PaperClip: A Decentralized Confidential Compute Marketplace for Privacy-Preserving Artificial Intelligence Workloads

arXiv Preprint — Draft for Review

Manthan Bhanushali
Independent Researcher
paperclip-research@example.com

March 12, 2026

Abstract— The rapid proliferation of machine learning and artificial intelligence has created unprecedented demand for scalable, high-throughput compute infrastructure. Yet the dominant paradigm—centralized cloud providers—introduces a fundamental tension between computational utility and data privacy. Organizations processing sensitive workloads such as electronic health records, financial transaction histories, biometric identifiers, or proprietary model weights must currently choose between leveraging powerful cloud infrastructure and maintaining control over their most sensitive assets.

This paper presents PaperClip, a decentralized confidential compute marketplace designed to eliminate this tension. PaperClip combines hardware-rooted trusted execution environments (TEEs), cryptographic remote attestation, encrypted workload submission, a multi-criteria distributed job scheduler, and a transparent economic marketplace to enable users to execute AI workloads on untrusted third-party hardware without revealing the content of those workloads to infrastructure operators.

The PaperClip system architecture decomposes into five principal components: a client-facing React.js marketplace interface, a node discovery and metadata registry, a cryptographic attestation verification service, a multi-criteria distributed job scheduler, and confidential execution nodes equipped with Intel SGX enclaves or AMD SEV-SNP virtual machines. Compute providers register their nodes by publishing hardware attestation reports and capability metadata. Users discover nodes through the marketplace, verify hardware integrity via remote attestation, encrypt workloads under the enclave's attested public key, and submit encrypted job payloads. The enclave decrypts and executes the workload in hardware-isolated memory, returns encrypted results, and the user decrypts locally. At no point does the infrastructure operator, network adversary, or privileged software stack observe plaintext workload data.

We present a complete system design, a formal cryptographic model, a detailed threat analysis, an economic incentive framework, and a thorough

Keywords: Confidential Computing, Trusted Execution Environments, Intel SGX, AMD SEV-SNP, Remote Attestation, Distributed Job Scheduling, Privacy-Preserving AI, Secure Enclaves, Decentralized Marketplace, GPU Confidential Computing, Encrypted Workloads, Homomorphic Encryption

1 Introduction

The global machine learning compute market is undergoing rapid structural change. As AI workloads grow in scale and strategic value, the computational demands for training large language models, running inference at scale, and processing sensitive datasets have consolidated around a handful of hyperscale cloud providers—primarily Amazon Web Services, Microsoft Azure, and Google Cloud Platform. This concentration creates a structural privacy problem: organizations with the most sensitive data are also the organizations most in need of powerful compute infrastructure, yet they cannot fully trust the infrastructure operators with their data.

Consider a hospital system seeking to fine-tune a clinical language model on patient records. The training data is legally protected under HIPAA and ethically bounded by patient consent, yet the hospital lacks the GPU infrastructure to run the fine-tuning locally. Similarly, a financial institution running transaction-level fraud detection on proprietary datasets cannot afford data exposure to a cloud provider that also serves competitors. A biotech firm with proprietary protein structure prediction pipelines faces the same dilemma: the compute exists in the cloud, but the model weights represent billions of dollars of research investment that cannot be safely handed to an untrusted operator.

Existing solutions are inadequate. Federated learning reduces data movement but does not eliminate it and introduces gradient leakage vulnerabilities [6]. Homomorphic encryption enables computation on ciphertext but incurs orders-of-magnitude overhead incompatible with large neural networks [7]. Secure multiparty computation achieves strong privacy guarantees but requires prohibitive communication rounds for deep learning workloads [8]. Centralized confidential cloud services such as Azure Confidential Computing and AWS Nitro Enclaves improve privacy posture but retain the fundamental trust assumption in the cloud provider's control plane.

PaperClip addresses this gap through a fundamentally different architecture. Rather than asking users to trust a centralized operator, PaperClip constructs a decentralized marketplace in which hardware-enforced privacy guarantees are verifiable by the user before any data is submitted. The core insight is that modern trusted execution environment (TEE) hardware—specifically Intel SGX, AMD SEV-SNP, and emerging GPU confidential computing architectures—provides cryptographically verifiable isolation that cannot be circumvented by the infrastructure operator, regardless of their privilege level. By combining TEE-based compute nodes with a decentralized marketplace protocol and a transparent economic model, PaperClip enables any provider with the right hardware to offer genuinely private compute, and any user to verify that privacy guarantee before submitting sensitive data.

1.1 Problem Statement

The central problem PaperClip addresses can be stated precisely: *How can a user delegate computation to an untrusted hardware operator and obtain a cryptographically verifiable assurance that the operator has not observed the input data, the computation, or the output, even with full administrative access to the host operating system and hypervisor?*

This formulation is stronger than traditional cloud security models, which protect against external attackers but not the cloud provider itself. It is also stronger than software-only privacy techniques, which require trusting the software stack rather than deriving guarantees from hardware.

1.2 Contributions

This paper makes the following research contributions:

1. **System Architecture:** We present a complete, end-to-end architecture for a decentralized confidential compute marketplace, including all protocol layers from user-facing API to enclave execution engine.

2. **Cryptographic Protocol:** We specify a formal key exchange and workload encryption protocol based on TEE remote attestation, providing end-to-end confidentiality between client and enclave.

3. **Distributed Scheduler:** We design a multi-criteria job scheduler with hardware-awareness, incorporating TEE type, attestation freshness, latency, price, and historical reliability.

4. **Economic Model:** We propose a marketplace incentive framework that aligns provider and

user incentives, incorporates energy-cost dynamics, and resists Sybil and collusion attacks.

5. **Prototype Implementation:** We describe a full-stack prototype including a React.js marketplace frontend, a Node.js/Express API backend, an attestation verification service, and mock enclave nodes, with source code analysis.

6. **Security Analysis:** We provide a formal threat model and argue confidentiality, integrity, and authenticity properties under specified adversary models.

1.3 Paper Organization

The remainder of this paper is structured as follows. Section 2 provides background on TEEs, remote attestation, and related privacy-preserving computation paradigms. Section 3 presents the PaperClip system architecture. Section 4 details the end-to-end confidential execution workflow. Section 5 specifies the TEE attestation protocol. Section 6 formalizes the cryptographic model. Section 7 defines the threat model. Section 8 analyzes security properties. Section 9 discusses GPU confidential computing. Section 10 describes the distributed job scheduler. Section 11 presents the economic marketplace model. Section 12 analyzes the prototype implementation with code. Section 13 proposes an evaluation methodology. Section 14 surveys related work. Section 15 discusses future work. Section 16 concludes.

2 Background

2.1 The Privacy Problem in AI Compute

Modern AI workloads are characterized by two properties that jointly create a privacy crisis: they require large amounts of sensitive data, and they demand massive parallelized hardware that almost no data owner possesses on-premises. The result is a forced dependency on cloud providers who sit in a position of total privilege over customer data during computation.

The traditional security model in cloud computing distinguishes between *data at rest* (protected by disk encryption) and *data in transit* (protected by TLS). Both are well-understood problems with mature solutions. The missing piece is *data in use*—data that must be decrypted into CPU registers and main memory to be processed. Conventional hardware provides no isolation between the privileged software layers (hypervisor, OS kernel) and the application. This means any sufficiently privileged actor on the host—a rogue employee, a compromised kernel, a co-tenant exploit, or the provider itself—can in principle read any application memory.

2.2 Trusted Execution Environments

Trusted Execution Environments are hardware-enforced isolated execution domains that protect the confidentiality and integrity of code and data even against privileged software adversaries. The key property distinguishing TEEs from conventional virtualization or containerization is that the isolation guarantees are *hardware-enforced*, backed by hardware keys that are physically embedded in the processor and inaccessible to any software, including the operating system, hypervisor, or BIOS.

2.2.1 Intel Software Guard Extensions (SGX)

Intel SGX [1] is a set of x86 instruction set extensions that allow applications to create *enclaves*: isolated memory regions called Enclave Page Cache (EPC) that are encrypted using keys held by on-chip hardware and inaccessible to all software outside the enclave boundary. SGX enclaves provide the following guarantees:

- **Memory Encryption:** EPC pages are encrypted by the Memory Encryption Engine (MEE) using AES-XTS with a 128-bit key generated on-die at boot and never exposed to software.

- **Memory Integrity:** A Merkle-tree-based memory integrity tree protects against replay and remapping attacks on EPC contents.

- **Isolation:** The processor enforces that enclave code and data cannot be accessed by the OS, hypervisor, or any other enclave, using hardware access control checks on every memory access.

- **Attestation:** The enclave can produce a *Quote*—a signed attestation report—that binds the enclave's identity (a cryptographic measurement called MRENCLAVE) to statements about its execution environment.

SGX v1 imposed a 128 MB EPC limit, which constrained AI workloads. SGX v2, available on Ice Lake and later Xeon processors, supports dynamic EPC management and larger enclave memory regions, making it more viable for inference workloads.

However, SGX remains CPU-only and cannot directly protect GPU computation.

2.2.2 AMD Secure Encrypted Virtualization (SEV-SNP)

AMD SEV-SNP [2] takes a VM-granularity approach to confidential computing. Rather than isolating a single application-level enclave, SEV-SNP protects an entire virtual machine from the hypervisor and other VMs. Key features include:

- **VM-Level Memory Encryption:** Each VM's memory is encrypted with a unique key generated by the AMD Secure Processor (SP), invisible to the hypervisor.
- **Integrity Protection:** SNP (Secure Nested Paging) adds hardware-enforced immutability to guest-owned memory pages, preventing the hypervisor from silently remapping or modifying guest memory.
- **Attestation:** The AMD SP generates signed attestation reports binding the VM's measurement (a hash over its launch state) to processor firmware version and security level.
- **Reverse Map Table (RMP):** A hardware-maintained table tracks ownership of all memory pages, enforcing that the hypervisor cannot access guest pages without cooperative exception handling.

SEV-SNP is particularly well-suited for PaperClip's node model because it protects whole containerized workloads at VM granularity rather than requiring re-architecting applications to use SGX-style enclave APIs.

2.2.3 ARM Confidential Compute Architecture (CCA)

ARM CCA [4] introduces *Realms*, hardware-isolated execution environments protected from the hypervisor by a new privilege level called the Realm Management Monitor (RMM). While primarily targeting mobile and embedded platforms, ARM CCA is increasingly relevant for edge AI workloads on ARM-based server clusters such as AWS Graviton.

2.2.4 NVIDIA Confidential Computing

NVIDIA's H100 GPU introduced the first production confidential GPU architecture [5]. It provides hardware-encrypted GPU memory accessible only to attested workloads, with a GPU attestation service enabling remote verification of GPU identity and firmware. This is critical for PaperClip's GPU confidential computing tier, discussed in detail in Section 9.

2.3 Remote Attestation

Remote attestation is the mechanism by which a relying party (in PaperClip's case, the user) can verify the identity and integrity of a remote enclave without physical access to the hardware. The attestation process produces a signed *attestation report* or *Quote* that contains:

1. A cryptographic measurement of the enclave code (MRENCLAVE for SGX, or a launch digest for SEV-SNP).
2. The enclave's public key, freshly generated inside the enclave at initialization.
3. Platform security information (firmware versions, patch levels).
4. A hardware signature from the processor's secret key, establishing that the report was produced by genuine Intel/AMD hardware.

Intel provides the Intel Trust Authority (ITA) service for SGX quote verification; AMD provides a similar attestation service. Decentralized verification approaches using on-chain attestation oracles have also been proposed [13].

2.4 Existing Privacy-Preserving Computation Paradigms

To contextualize PaperClip's design, we briefly survey alternative approaches to privacy-preserving computation.

Homomorphic Encryption (HE): HE [34] allows arbitrary computation on encrypted data without decryption. Fully homomorphic encryption (FHE) achieves this at the cost of enormous computation overhead—typically 6–8 orders of magnitude slower than plaintext computation. While recent TFHE and CKKS improvements have closed this gap somewhat [35], FHE remains impractical for large-scale AI training.

Secure Multi-Party Computation (MPC): MPC [36] allows multiple parties to jointly compute a function without revealing their individual inputs. SecureML [8] demonstrated privacy-preserving linear regression and neural network training using

MPC. However, communication complexity scales poorly with model depth.

Differential Privacy (DP): DP [9] adds calibrated noise to prevent inference of individual records from model outputs. DP protects training data privacy but not model confidentiality, and the accuracy-privacy tradeoff is poorly suited to many medical and financial applications.

Federated Learning (FL): FL [10] keeps raw data on device, sharing only gradient updates. Gradient inversion attacks [6] and membership inference attacks [11] demonstrate that FL alone does not provide strong privacy guarantees.

PaperClip complements and can incorporate these techniques: workloads submitted to PaperClip could be differentially private training jobs or MPC protocol participants, further layering privacy guarantees.

3 System Architecture

3.1 Architectural Overview

PaperClip is organized as a five-tier distributed system, illustrated conceptually below. Each tier serves a distinct functional role and interacts with adjacent tiers through well-defined interfaces.

1. **Client Interface Tier:** A React.js single-page application that presents the marketplace, node discovery UI, job submission workflow, and result retrieval interface to users.

2. **API Gateway Tier:** A Node.js/Express REST API that mediates all communication between the client and backend services, enforcing authentication, rate limiting, and request validation.

3. **Registry and Attestation Tier:** A node discovery registry that stores provider metadata and a cryptographic attestation verification service that validates TEE quotes before nodes are admitted to the marketplace.

4. **Scheduler Tier:** A multi-criteria job scheduler that receives job submissions, matches them to suitable nodes, manages the job queue, and monitors execution state.

5. **Compute Node Tier:** Physical or virtual machines operated by compute providers, equipped with TEE hardware, running the PaperClip node agent that manages enclave lifecycle and job execution.

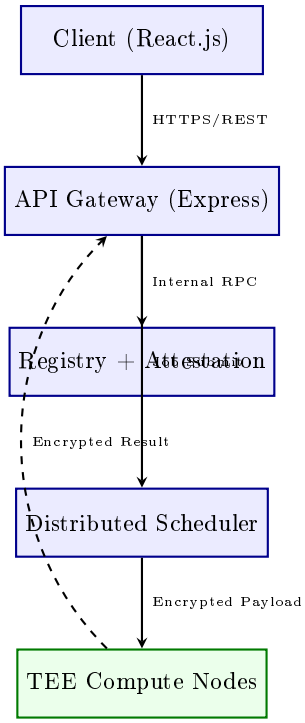

Figure 1: PaperClip five-tier architecture. Arrows represent primary data flow directions. The dashed line indicates the encrypted result return path from compute node to client via the API.

3.2 Client Interface Tier

The client interface is a React.js application implementing a marketplace dashboard, node browser, job submission wizard, and result viewer. It communicates exclusively with the API gateway over HTTPS, never directly with compute nodes. The client is responsible for:

- User authentication via JWT tokens
- Node browsing and filtering by hardware type, price, geographic region, and attested security level
- Attestation report display and verification UI
- Workload encryption using the node's attested public key before upload
- Job status polling and result decryption

The React application uses a context-provider architecture to manage global application state. The authentication context provides token management and session handling:

```jsx
// src/context/AuthContext.jsx
import React, { createContext, useContext,
  useState, useCallback, useEffect } from 'react';
import { apiClient } from '../api/client';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(
    () => localStorage.getItem('paperclip_token')
  );
  const [loading, setLoading] = useState(true);

  // Validate stored token on mount
  useEffect(() => {
    if (token) {
      apiClient.defaults.headers.common[
        'Authorization'
      ] = `Bearer ${token}`;
      apiClient.get('/auth/me')
        .then(res => setUser(res.data.user))
        .catch(() => {
          setToken(null);
          localStorage.removeItem('paperclip_token');
        })
        .finally(() => setLoading(false));
    } else {
      setLoading(false);
    }
  }, [token]);

  const login = useCallback(async (email, password) => {
    const res = await apiClient.post('/auth/login',
      { email, password });
    const { token: newToken, user: userData } = res.data;
    localStorage.setItem('paperclip_token', newToken);
    apiClient.defaults.headers.common[
      'Authorization'
    ] = `Bearer ${newToken}`;
    setToken(newToken);
    setUser(userData);
    return userData;
  }, []);

  const logout = useCallback(() => {
    localStorage.removeItem('paperclip_token');
    delete apiClient.defaults.headers.common['Authorization'];
    setToken(null);
    setUser(null);
  }, []);

  return (
    <AuthContext.Provider value={{
      user, token, loading, login, logout
    }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

Listing 1: Authentication context provider managing JWT lifecycle and API session state.

This context pattern is architecturally significant: by centralizing token management and API header injection in a single provider, all child components gain authenticated API access without prop-drilling credential state. The `useEffect` hook on mount validates any persisted token against the `/auth/me` endpoint, ensuring stale or revoked tokens are cleared automatically. The `useCallback` memoization prevents unnecessary re-renders of child components that depend on `login` or `logout`.

The application routing structure reflects the marketplace workflow:

```jsx
// src/App.jsx
import React, { Suspense, lazy } from 'react';
import { BrowserRouter, Routes, Route,
  Navigate } from 'react-router-dom';
import { AuthProvider, useAuth } from './context/AuthContext';
import LoadingSpinner from './components/LoadingSpinner';

// Lazy-loaded route components for code splitting
const Landing     = lazy(() => import('./pages/Landing'));
const Login       = lazy(() => import('./pages/Login'));
const Register    = lazy(() => import('./pages/Register'));
const Dashboard   = lazy(() => import('./pages/Dashboard'));
const NodeBrowser = lazy(() => import('./pages/NodeBrowser'));
const NodeDetail  = lazy(() => import('./pages/NodeDetail'));
const SubmitJob   = lazy(() => import('./pages/SubmitJob'));
const JobStatus   = lazy(() => import('./pages/JobStatus'));
const JobResult   = lazy(() => import('./pages/JobResult'));
const ProviderReg = lazy(() => import('./pages/ProviderRegister'));

function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();
  if (loading) return <LoadingSpinner />;
  return user ? children : <Navigate to="/login" replace />;
}

function ProviderRoute({ children }) {
```

```
  const { user, loading } = useAuth();
  if (loading) return <LoadingSpinner />;
  if (!user) return <Navigate to="/login" replace />;
  if (user.role !== 'provider')
    return <Navigate to="/dashboard" replace />;
  return children;
}

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Suspense fallback={<LoadingSpinner />}>
          <Routes>
            <Route path="/"          element={<Landing />} />
            <Route path="/login"     element={<Login />} />
            <Route path="/register"  element={<Register />} />
            <Route path="/dashboard" element={
              <ProtectedRoute><Dashboard /></ProtectedRoute>
            }/>
            <Route path="/nodes"     element={
              <ProtectedRoute><NodeBrowser /></ProtectedRoute>
            }/>
            <Route path="/nodes/:nodeId" element={
              <ProtectedRoute><NodeDetail /></ProtectedRoute>
            }/>
            <Route path="/jobs/submit/:nodeId" element={
              <ProtectedRoute><SubmitJob /></ProtectedRoute>
            }/>
            <Route path="/jobs/:jobId" element={
              <ProtectedRoute><JobStatus /></ProtectedRoute>
            }/>
            <Route path="/jobs/:jobId/result" element={
              <ProtectedRoute><JobResult /></ProtectedRoute>
            }/>
            <Route path="/provider/register" element={
              <ProviderRoute><ProviderReg /></ProviderRoute>
            }/>
          </Routes>
        </Suspense>
      </BrowserRouter>
    </AuthProvider>
  );
}
```

Listing 2: Top-level application routing structure mapping URL paths to marketplace workflow stages.

This routing topology maps directly to the confidential execution workflow. A user navigates `/nodes` to browse available TEE nodes, selects a node at `/nodes/:nodeId` to inspect its attestation report, proceeds to `/jobs/submit/:nodeId` to encrypt and submit a workload, monitors execution at `/jobs/:jobId`, and retrieves decrypted results at `/jobs/:jobId/result`. The lazy-loaded code splitting ensures that cryptographic libraries used on the job submission page (Section 6) are not bundled into the initial load.

3.3 API Gateway Tier

The API gateway is a Node.js application using the Express framework, providing RESTful endpoints for all client-backend communication. It is organized around four domain areas:

1. **Authentication:** `/api/v1/auth` — user registration, login, token refresh, session validation
2. **Nodes:** `/api/v1/nodes` — node listing, filtering, detail retrieval, attestation report access
3. **Jobs:** `/api/v1/jobs` — job submission, status polling, result retrieval, job cancellation
4. **Providers:** `/api/v1/providers` — provider registration, node registration, revenue reporting

The API client on the frontend side abstracts all HTTP communication:

```
// src/api/client.js
import axios from 'axios';

export const apiClient = axios.create({
  baseURL: process.env.REACT_APP_API_URL
    || 'http://localhost:4000/api/v1',
  timeout: 30000,
  headers: { 'Content-Type': 'application/json' }
});

// Response interceptor: normalize errors
apiClient.interceptors.response.use(
  response => response,
  error => {
    const normalized = {
      message: error.response?.data?.message
        || error.message || 'Unknown error',
      status:  error.response?.status || 0,
      code:    error.response?.data?.code || 'UNKNOWN'
    };
    // Auto-logout on 401 Unauthorized
    if (normalized.status === 401) {
      localStorage.removeItem('paperclip_token');
      window.location.href = '/login';
```

```
    }
    return Promise.reject(normalized);
  }
);

// Domain-specific API methods
export const nodesApi = {
  list:       (filters) =>
    apiClient.get('/nodes', { params: filters }),
  getById:    (id) =>
    apiClient.get(`/nodes/${id}`),
  getAttestation: (id) =>
    apiClient.get(`/nodes/${id}/attestation`),
  getCapabilities: (id) =>
    apiClient.get(`/nodes/${id}/capabilities`)
};

export const jobsApi = {
  submit:     (payload) =>
    apiClient.post('/jobs', payload),
  getStatus:  (jobId) =>
    apiClient.get(`/jobs/${jobId}`),
  getResult:  (jobId) =>
    apiClient.get(`/jobs/${jobId}/result`),
  cancel:     (jobId) =>
    apiClient.delete(`/jobs/${jobId}`),
  listByUser: (userId, opts) =>
    apiClient.get(`/users/${userId}/jobs`, { params: opts })
};

export const providerApi = {
  register:      (data) =>
    apiClient.post('/providers', data),
  registerNode:  (providerId, nodeData) =>
    apiClient.post(`/providers/${providerId}/nodes`, nodeData),
  getEarnings:   (providerId) =>
    apiClient.get(`/providers/${providerId}/earnings`)
};
```

Listing 3: Centralized API client with request/response interceptors for authentication and error normalization.

The `apiClient` design pattern provides several architectural benefits. The centralized interceptor handles authentication headers, enforces automatic logout on 401 responses, and normalizes error objects to a consistent shape, reducing error handling code duplication across the application. The domain-specific API objects group related operations, improving code organization and testability.

3.4 Node Discovery Registry

The node discovery registry is a persistent data store (PostgreSQL with a Redis caching layer) that maintains the authoritative record of all compute providers and their compute nodes. Each node record contains:

- **Node identity:** UUID, provider ID, public key (enclave-generated), geographic region, network endpoint
- **Hardware metadata:** TEE type (SGX/SEV-SNP/CCA), TEE version, CPU model, GPU model (if present), available EPC memory
- **Attestation record:** Latest attestation quote, verification timestamp, trust score
- **Capability metadata:** Supported workload types, maximum job duration, memory limits
- **Economic metadata:** Price per compute unit, pricing model (per-second, per-job, auction-based), current availability
- **Reputation:** Historical job completion rate, median execution time, user ratings

The registry exposes a filterable list endpoint allowing users to discover nodes matching their requirements:

```
// server/routes/nodes.js
const express = require('express');
const router  = express.Router();
const { authenticate } = require('../middleware/auth');
const NodeService = require('../services/NodeService');
const AttestationService =
  require('../services/AttestationService');

/**
 * GET /api/v1/nodes
 * Returns paginated, filtered list of available compute nodes.
 * Filters: teeType, region, minMemoryGB, maxPricePerHour,
 *          hasGpu, attestedWithin (seconds), sort
 */
router.get('/', authenticate, async (req, res) => {
  try {
    const {
      teeType, region, minMemoryGB, maxPricePerHour,
      hasGpu, attestedWithin = 3600,
      sort = 'price_asc', page = 1, limit = 20
    } = req.query;

    const filters = {
      teeType:          teeType || null,
      region:           region  || null,
      minMemoryGB:      minMemoryGB  ? Number(minMemoryGB)  : null,
      maxPricePerHour:  maxPricePerHour
                          ? Number(maxPricePerHour) : null,
```

```
      hasGpu:           hasGpu === 'true',
      attestedWithin:   Number(attestedWithin),
      isActive:         true
    };

    const nodes = await NodeService.listNodes(
      filters, sort,
      { page: Number(page), limit: Number(limit) }
    );

    // Enrich each node with fresh attestation status
    const enriched = await Promise.all(
      nodes.items.map(async node => ({
        ...node,
        attestationStatus:
          await AttestationService.getStatus(node.id),
        trustScore:
          await NodeService.computeTrustScore(node.id)
      }))
    );

    res.json({
      nodes: enriched,
      pagination: {
        page: nodes.page,
        limit: nodes.limit,
        total: nodes.total,
        pages: Math.ceil(nodes.total / nodes.limit)
      }
    });
  } catch (err) {
    res.status(500).json({
      message: 'Registry query failed',
      error: err.message
    });
  }
});

/**
 * GET /api/v1/nodes/:nodeId/attestation
 * Returns the full attestation report for a specific node.
 */
router.get('/:nodeId/attestation', authenticate,
  async (req, res) => {
  try {
    const report = await AttestationService
      .getFullReport(req.params.nodeId);
    if (!report)
      return res.status(404).json({
        message: 'Node not found'
      });
    res.json(report);
  } catch (err) {
    res.status(500).json({
      message: 'Attestation retrieval failed',
      error: err.message
    });
  }
});

module.exports = router;
```

Listing 4: Node registry listing endpoint with multi-criteria filtering on the Express backend.

3.5 Attestation Verification Service

The attestation verification service is a critical security component that validates TEE attestation quotes before nodes are admitted to the registry and before users encrypt workloads for a specific node. It integrates with vendor attestation services—Intel Trust Authority for SGX, AMD Attestation Service for SEV-SNP—and performs the following verification steps:

1. **Quote structural integrity:** Parse and validate quote format, version, and field constraints

2. **Signature verification:** Verify the hardware signature using the processor's certificate chain, anchored at the vendor root CA

3. **MRENCLAVE/Measurement check:** Compare the enclave measurement in the quote against the expected value for the current PaperClip node agent version

4. **Security level assertion:** Check that no known vulnerabilities affect the reported processor firmware version (using Intel PCCS or AMD equivalent)

5. **Freshness:** Ensure the quote timestamp is within an acceptable staleness window

6. **Public key binding:** Extract and record the enclave's attested public key for use in workload encryption

3.6 Scheduler Tier

The scheduler receives submitted jobs from the API gateway and is responsible for matching each job to the optimal available compute node. The scheduler maintains a priority queue of pending jobs and an in-memory table of node availability state. Job matching runs as a continuous loop, periodically re-evaluating queued jobs against the current node availability landscape. The scheduler's design is described in detail in Section 10.

3.7 Compute Node Tier

Compute nodes are operated by providers who have registered with the marketplace. Each node runs the PaperClip Node Agent, a daemon process responsible for:

- Maintaining an active TEE enclave (SGX enclave process or SEV-SNP protected VM)
- Generating and publishing fresh attestation quotes at configurable intervals
- Receiving encrypted job payloads from the scheduler
- Passing encrypted payloads into the enclave via secure IPC
- Monitoring enclave execution and reporting status to the scheduler
- Returning encrypted results to the API gateway

The enclave itself runs a minimal trusted runtime—a stripped-down Python or WebAssembly interpreter for SGX, or a containerized execution environment for SEV-SNP—that receives, decrypts, and executes workload code entirely within hardware-protected memory.

4 Confidential Execution Workflow

The end-to-end confidential execution workflow comprises five sequential steps, each building on the cryptographic and trust guarantees of its predecessor. We describe each step in full detail.

4.1 Step 1: Node Discovery and Browsing

A user begins by browsing the node marketplace. The client application calls `GET /api/v1/nodes` with the user's filtering criteria. The registry queries its database for active nodes matching the criteria and returns a paginated list enriched with current attestation status and computed trust scores.

```python
// src/pages/NodeBrowser.jsx
import React, { useState, useEffect, useCallback }
  from 'react';
import { useNavigate } from 'react-router-dom';
import { nodesApi } from '../api/client';
import NodeCard from '../components/NodeCard';
import FilterPanel from '../components/FilterPanel';

export default function NodeBrowser() {
  const [nodes,   setNodes]   = useState([]);
  const [loading, setLoading] = useState(true);
  const [error,   setError]   = useState(null);
  const [filters, setFilters] = useState({
    teeType: '', region: '', minMemoryGB: '',
    maxPricePerHour: '', hasGpu: false,
    attestedWithin: 3600
  });
  const [page,    setPage]    = useState(1);
  const [total,   setTotal]   = useState(0);
  const navigate = useNavigate();

  const fetchNodes = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      const res = await nodesApi.list({
        ...filters, page, limit: 20
      });
      setNodes(res.data.nodes);
      setTotal(res.data.pagination.total);
    } catch (e) {
      setError(e.message);
    } finally {
      setLoading(false);
    }
  }, [filters, page]);

  useEffect(() => { fetchNodes(); }, [fetchNodes]);

  const handleNodeSelect = (nodeId) => {
    navigate(`/nodes/${nodeId}`);
  };

  return (
    <div className="node-browser">
      <h1>Available Compute Nodes</h1>
      <FilterPanel
        filters={filters}
        onChange={setFilters}
        onReset={() => setFilters({
          teeType: '', region: '',
          minMemoryGB: '', maxPricePerHour: '',
          hasGpu: false, attestedWithin: 3600
        })}
      />
      {loading && <div className="spinner"/>}
      {error   && <div className="error">{error}</div>}
      <div className="node-grid">
        {nodes.map(node => (
          <NodeCard
            key={node.id}
            node={node}
            onSelect={() => handleNodeSelect
```

```
    (node.id)}
          />
        ))}
      </div>
      {total > 20 && (
        <Pagination
          page={page}
          totalPages={Math.ceil(total/20)}
          onChange={setPage}
        />
      )}
    </div>
  );
}
```

Listing 5: Node browser component displaying attested compute nodes with hardware metadata and trust indicators.

4.2 Step 2: Attestation Verification

Before submitting any workload, the user must verify the selected node's attestation report. The NodeDetail page fetches the full attestation report and presents it to the user in an interpretable form.

```
// src/pages/NodeDetail.jsx
import React, { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { nodesApi } from '../api/client';
import AttestationPanel from
  '../components/AttestationPanel';
import CapabilitiesPanel from
  '../components/CapabilitiesPanel';

export default function NodeDetail() {
  const { nodeId } = useParams();
  const navigate   = useNavigate();
  const [node,        setNode]        = useState(null);
  const [attestation, setAttestation] = useState(null);
  const [capabilities,setCapabilities]= useState(null);
  const [loading,     setLoading]     = useState(true);

  useEffect(() => {
    setLoading(true);
    Promise.all([
      nodesApi.getById(nodeId),
      nodesApi.getAttestation(nodeId),
      nodesApi.getCapabilities(nodeId)
    ]).then(([n, a, c]) => {
      setNode(n.data);
      setAttestation(a.data);
      setCapabilities(c.data);
    }).finally(() => setLoading(false));
  }, [nodeId]);

  if (loading) return <div className="spinner" />;

  return (
    <div className="node-detail">
      <h1>{node.name}</h1>
      <div className="node-meta">
        <span>Region: {node.region}</span>
        <span>TEE Type: {node.teeType}</span>
        <span>Price: ${node.pricePerHour}/hr</span>
      </div>

      {/* Attestation panel shows enclave measurements,
          hardware signatures, and trust chain */}
      <AttestationPanel
        attestation={attestation}
        nodePublicKey={node.enclavePublicKey}
      />

      <CapabilitiesPanel capabilities={capabilities} />

      <button
        className="btn-primary"
        onClick={() => navigate(
          `/jobs/submit/${nodeId}`,
          { state: { node, attestation } }
        )}
        disabled={
          attestation?.status !== 'VERIFIED'
        }
      >
        {attestation?.status === 'VERIFIED'
          ? 'Submit Job to This Node'
          : 'Node attestation unverified cannot proceed'}
      </button>
    </div>
  );
}
```

Listing 6: Node detail page fetching and presenting TEE attestation report for user verification.

This design enforces a critical security invariant at the UI level: the "Submit Job" button is disabled unless the attestation service has returned a `VERIFIED` status for the selected node. This prevents accidental submission to unattested nodes, though the API gateway enforces the same check server-side as a defense-in-depth measure.

4.3 Step 3: Workload Encryption and Submission

Once the user has verified the node's attestation, they proceed to job submission. The critical security property of this step is that workload data is encrypted *client-side* using the node's attested public key *before* it leaves the user's browser. The API gateway and all intermediate network hops see only ciphertext.

```
// src/utils/crypto.js
import * as jose from 'jose';

/**
 * Encrypts a workload payload under the enclave's
 * attested public key using ECDH-ES + AES-256-GCM.
 * The private key never leaves the enclave hardware.
 *
 * @param {object} workload - Plaintext job payload
 * @param {string} enclavePublicKeyPem - PEM-encoded
 *   enclave EC public key (P-384) from attestation report
 * @param {string} jobNonce - Fresh nonce from server
 *   to prevent replay attacks
 * @returns {string} JWE compact serialization
 */
export async function encryptWorkload(
  workload, enclavePublicKeyPem, jobNonce
) {
  // Parse the enclave's public key
  const enclaveKey = await jose.importSPKI(
    enclavePublicKeyPem,
    'ECDH-ES'
  );

  // Encode payload with nonce for replay protection
  const plaintext = new TextEncoder().encode(
    JSON.stringify({
      ...workload,
      nonce: jobNonce,
      clientTimestamp: Date.now()
    })
  );

  // ECDH-ES key agreement + AES-256-GCM content encryption
  const jwe = await new jose.CompactEncrypt(plaintext)
    .setProtectedHeader({
      alg: 'ECDH-ES',
      enc: 'A256GCM',
      kid: workload.targetNodeId
    })
    .encrypt(enclaveKey);

  return jwe;
}

/**
 * Decrypts a result JWE using the user's private key.
 * The enclave encrypts results under the user's
 * ephemeral public key provided in the job submission.
 */
export async function decryptResult(
  resultJwe, userPrivateKeyJwk
) {
  const userPrivKey = await jose.importJWK(
    userPrivateKeyJwk,
    'ECDH-ES'
  );
  const { plaintext } = await jose.compactDecrypt(
    resultJwe, userPrivKey
  );
  return JSON.parse(new TextDecoder().decode(plaintext));
}

/**
 * Generates an ephemeral EC key pair for the user.
 * The public key is sent to the enclave (in the encrypted
 * payload) so the enclave can encrypt results back.
 */
export async function generateEphemeralKeyPair() {
  const { publicKey, privateKey } =
    await crypto.subtle.generateKey(
      { name: 'ECDH', namedCurve: 'P-384' },
      true, // extractable
      ['deriveKey', 'deriveBits']
    );
  const publicJwk  = await crypto.subtle.exportKey(
    'jwk', publicKey);
  const privateJwk = await crypto.subtle.exportKey(
    'jwk', privateKey);
  return { publicJwk, privateJwk };
}
```

Listing 7: Workload encryption using ECIES with the enclave's attested public key, executed entirely client-side before transmission.

The job submission component orchestrates the full client-side encryption workflow:

```
// src/pages/SubmitJob.jsx
import React, { useState } from 'react';
import { useParams, useLocation, useNavigate }
  from 'react-router-dom';
import { jobsApi } from '../api/client';
import {
  encryptWorkload, generateEphemeralKeyPair
} from '../utils/crypto';
import WorkloadEditor from '../components/WorkloadEditor';

export default function SubmitJob() {
  const { nodeId }          = useParams();
  const { state }           = useLocation();
    // node, attestation
  const navigate            = useNavigate();
  const [workload,   setWorkload]   = useState('');
  const [submitting, setSubmitting] = useState(false);
  const [error,      setError]      = useState(null);
```

```
  const [step,        setStep]        =
    useState('edit');

  const handleSubmit = async () => {
    setSubmitting(true);
    setError(null);
    try {
      setStep('keygen');
      // Generate ephemeral key pair for
    result encryption
      const { publicJwk, privateJwk } =
        await generateEphemeralKeyPair();

      setStep('nonce');
      // Fetch a fresh nonce from server to
    bind job
      const nonceRes = await jobsApi.
    getNonce(nodeId);
      const { nonce, jobId } = nonceRes.data
    ;

      setStep('encrypting');
      // Encrypt workload under enclave's
    attested public key
      const encryptedPayload = await
    encryptWorkload(
        {
          code:            workload,
          resultPublicKey: publicJwk, //
    enclave uses this
          targetNodeId:    nodeId,
          maxRunSeconds:   300
        },
        state.attestation.enclavePublicKey,
        nonce
      );

      setStep('submitting');
      // Submit ciphertext only     API sees
     no plaintext
      await jobsApi.submit({
        jobId,
        nodeId,
        encryptedPayload,
        // Store encrypted private key
    locally for result
        // decryption     never sent to
    server
      });

      // Store private key in sessionStorage
     for result page
      sessionStorage.setItem(
        `result_key_${jobId}`,
        JSON.stringify(privateJwk)
      );

      navigate(`/jobs/${jobId}`);
    } catch (e) {
      setError(e.message || 'Submission
    failed');
      setStep('edit');
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <div className="submit-job">
      <h1>Submit Confidential Job</h1>
      <div className="target-node">
        <strong>Node:</strong> {state?.node
    ?.name}
        <span className="verified-badge">
          {state?.attestation?.status === '
    VERIFIED'
            ? '    Attested' : '
    Unverified'}
        </span>
      </div>
      <WorkloadEditor
        value={workload}
        onChange={setWorkload}
      />
      {submitting && (
        <div className="progress">
          {step === 'keygen'     && '
    Generating key pair...'}
          {step === 'nonce'      && '
    Fetching job nonce...'}
          {step === 'encrypting' &&
            'Encrypting under enclave public
     key...'}
          {step === 'submitting' &&
            'Submitting encrypted payload...
    '}
        </div>
      )}
      {error && <div className="error">{
    error}</div>}
      <button
        className="btn-primary"
        onClick={handleSubmit}
        disabled={submitting || !workload.
    trim()}
      >
        Encrypt and Submit
      </button>
    </div>
  );
}
```

Listing 8: Job submission page implementing client-side encryption and signed job payload construction.

The `handleSubmit` function implements the complete client-side security workflow. The ephemeral key pair generation ensures that each job has a unique encryption key, preventing result ciphertext from being linked across jobs. The nonce fetched from the server binds the encrypted payload to a specific job ID, defeating replay attacks. The private key is stored in `sessionStorage`—browser-tab-scoped, never sent to the server—ensuring that only the submitting browser can decrypt the result.

4.4 Step 4: Secure Enclave Execution

Upon receiving an encrypted job payload, the PaperClip Node Agent passes it into the enclave via a secure ECALL (SGX) or an encrypted channel to

the protected VM runtime (SEV-SNP). Inside the enclave, the execution sequence is:

1. **Payload Decryption:** The enclave uses its private key (generated at enclave initialization, stored only in EPC memory) to derive the symmetric content encryption key and decrypt the AES-256-GCM ciphertext.

2. **Nonce Verification:** The enclave checks that the nonce in the decrypted payload matches the expected value for this job, preventing replayed submissions.

3. **Workload Validation:** The enclave validates the workload structure, enforces resource limits (maximum execution time, memory budget), and checks that the workload type is permitted.

4. **Isolated Execution:** The workload code runs within a sandboxed interpreter (e.g., a compiled WASM module verified against an expected hash, or a restricted Python sandbox) inside the enclave.

5. **Result Encryption:** The execution output is encrypted under the user's `resultPublicKey` (the ephemeral public key embedded in the decrypted job payload), producing a JWE that only the user's browser can decrypt.

6. **Status Report:** The enclave signs a status message confirming successful execution and the result ciphertext hash, allowing the user to verify result integrity.

4.5 Step 5: Encrypted Result Return and Decryption

The enclave outputs the encrypted result JWE, which the Node Agent returns to the API gateway. The job result endpoint stores the ciphertext associated with the job ID. When the user polls for results:

```
// src/pages/JobResult.jsx
import React, { useState, useEffect } from 'react';
import { useParams } from 'react-router-dom';
import { jobsApi } from '../api/client';
import { decryptResult } from '../utils/crypto';

export default function JobResult() {
  const { jobId } = useParams();
  const [status, setStatus] = useState('polling');
  const [decryptedResult, setResult] = useState(null);
  const [error, setError] = useState(null);

  useEffect(() => {
    let intervalId;
    const pollStatus = async () => {
      try {
        const res = await jobsApi.getStatus(jobId);
        if (res.data.status === 'COMPLETE') {
          clearInterval(intervalId);
          setStatus('decrypting');

          // Retrieve private key from sessionStorage
          const privateJwk = JSON.parse(
            sessionStorage.getItem(`result_key_${jobId}`)
          );
          if (!privateJwk) {
            setError('Result key not found.' +
              'Cannot decrypt on a different device.');
            return;
          }

          // Fetch encrypted result
          const resultRes = await jobsApi.getResult(jobId);

          // Decrypt entirely client-side
          const plainResult = await decryptResult(
            resultRes.data.encryptedResult,
            privateJwk
          );

          // Clean up key from sessionStorage
          sessionStorage.removeItem(`result_key_${jobId}`);
          setResult(plainResult);
          setStatus('done');
        } else if (res.data.status === 'FAILED') {
          clearInterval(intervalId);
          setStatus('failed');
          setError(res.data.errorMessage);
        }
      } catch (e) {
        setError(e.message);
      }
    };

    intervalId = setInterval(pollStatus, 3000);
    pollStatus(); // immediate first poll
    return () => clearInterval(intervalId);
  }, [jobId]);

  return (
    <div className="job-result">
      {status === 'polling' &&
        <div>Waiting for enclave execution ...</div>}
      {status === 'decrypting' &&
```

```
        <div>Decrypting result...</div>}
      {status === 'done' && (
        <pre>{JSON.stringify(decryptedResult, null, 2)}</pre>
      )}
      {status === 'failed' && 
        <div className="error">Job failed: {error}</div>}
      </div>
  );
}
```

Listing 9: Job result page polling for completion and performing client-side decryption of returned ciphertext.

5 TEE Attestation Protocol

5.1 Attestation Goals

The attestation protocol serves three primary goals:

1. **Hardware Authenticity:** Prove that the remote machine contains genuine TEE hardware from a trusted manufacturer (Intel/AMD/NVIDIA), not an emulation or simulation.
2. **Software Integrity:** Prove that the TEE is running the correct, unmodified PaperClip Node Agent code—specifically, the code that will correctly enforce workload isolation and encryption.
3. **Key Binding:** Bind the enclave's public key to the attested hardware identity, so the user knows that encrypting data under that key guarantees it will only be decrypted inside the genuine, unmodified enclave.

5.2 Intel SGX Attestation Protocol

The Intel SGX attestation protocol [12] uses the Elliptic Curve Digital Signature Algorithm (ECDSA) to sign attestation quotes, with a certificate hierarchy anchored at Intel's root CA.

Step 1 – Attestation Request: The user client requests the attestation report for a specific node by calling `GET /api/v1/nodes/:nodeId/attestation`.

Step 2 – Quote Generation Request: The API gateway instructs the Node Agent to generate a fresh SGX Quote.

Step 3 – EREPORT and Quote Production: The enclave calls `EREPORT` to produce a hardware-signed report (REPORT structure) containing MRENCLAVE (hash of enclave code), MRSIGNER (hash of the signing key used to build the enclave), ISVPRODID, ISVSVN, and a `REPORTDATA` field that the enclave populates with its freshly generated public key hash. The Quoting Enclave (QE), a special Intel-provisioned enclave, converts the REPORT into an ECDSA-P256-signed Quote.

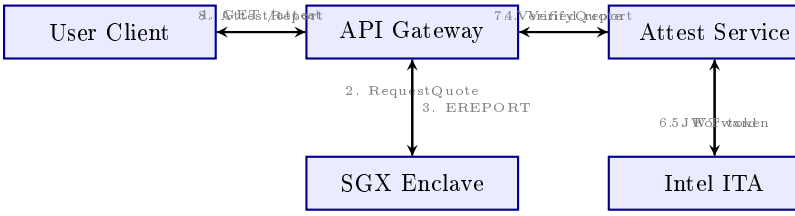

Figure 2: SGX ECDSA attestation message flow. Steps 1–8 show the full round-trip from user attestation request to verified report delivery.

Step 4–6 – Quote Verification via Intel Trust Authority: The attestation verification service sends the Quote to Intel Trust Authority, which verifies the ECDSA signature against Intel's PKI, checks the platform security level (TCB level), and returns a signed JWT token asserting the quote's validity.

Step 7–8 – Report Delivery: The verified report is returned to the user, who checks that MRENCLAVE matches the expected value for the current PaperClip node agent version and extracts the attested public key from the REPORTDATA field.

The attestation service implementation that handles this protocol:

```
// server/services/AttestationService.js
const axios  = require('axios');
const crypto = require('crypto');
const jose   = require('jose');

const EXPECTED_MRENCLAVES = new Map([
  ['v1.2.0', 'a3b1c2d4e5f6...'], // SHA-256 of enclave image
  ['v1.3.0', 'f7e6d5c4b3a2...'],
]);

const ITA_BASE =
  'https://api.trustedservices.intel.com/sgx/certification/v4';

class AttestationService {
  /**
   * Verify an SGX ECDSA quote using Intel Trust Authority.
   * Returns a verification record with trust level.
   */
  async verifySGXQuote(nodeId, rawQuoteHex) {
    const rawQuote = Buffer.from(rawQuoteHex, 'hex');

    // Step 1: Fetch PCK certificate and CRL
```

```
 from PCCS
    const pckCertRes = await axios.post(
      `${ITA_BASE}/quote`,
      { isvEnclaveQuote: rawQuote.toString('base64') },
      { headers: {
        'Ocp-Apim-Subscription-Key':
          process.env.INTEL_API_KEY
      }}
    );

    // Step 2: Parse and validate the JWT response
    const token = pckCertRes.headers[
      'x-iasreport-signature'
    ];
    const report = JSON.parse(
      Buffer.from(
        pckCertRes.data.split('.')[1], 'base64'
      ).toString()
    );

    // Step 3: Extract MRENCLAVE from quote body
    // SGX quote format: header(48) + body (384)
    // MRENCLAVE is at bytes 112-143 of the quote body
    const quoteBody   = rawQuote.slice(48);
    const mrenclave   = quoteBody.slice(112, 144)
                          .toString('hex');
    const reportData  = quoteBody.slice(320, 384)
                          .toString('hex');

    // Step 4: Validate MRENCLAVE against allowlist
    const expectedVersion = this
      .findVersionByMrenclave(mrenclave);
    if (!expectedVersion) {
      return {
        status: 'REJECTED',
        reason: 'MRENCLAVE not in allowlist',
        mrenclave
      };
    }

    // Step 5: Extract enclave public key from REPORTDATA
    // First 64 bytes of REPORTDATA = SHA-512 of enclave pubkey
    const pubKeyHash = reportData.slice(0, 128);

    // Step 6: Verify TCB level is acceptable
    const tcbStatus = report.tcbEvaluationDataNumber;
    if (['OUT_OF_DATE',
         'REVOKED'].includes(tcbStatus)) {
      return {
        status: 'REJECTED',
        reason: `Unacceptable TCB: ${tcbStatus}`
      };
    }

    // Record successful verification
    await this.storeVerification(nodeId, {
      mrenclave,
      version: expectedVersion,
      pubKeyHash,
      tcbStatus,
      verifiedAt: new Date().toISOString(),
      quoteId: pckCertRes.data.id
    });

    return {
      status:   'VERIFIED',
      mrenclave,
      version:  expectedVersion,
      tcbStatus,
      verifiedAt: new Date().toISOString()
    };
  }

  findVersionByMrenclave(mrenclave) {
    for (const [ver, expected] of EXPECTED_MRENCLAVES) {
      if (expected === mrenclave) return ver;
    }
    return null;
  }
}

module.exports = new AttestationService();
```

Listing 10: Server-side attestation verification service integrating Intel Trust Authority for SGX quote validation.

5.3 AMD SEV-SNP Attestation

AMD SEV-SNP attestation follows a conceptually similar protocol but operates at VM granularity. The attestation report is generated by the AMD Secure Processor (ASP), a dedicated on-die security processor, and signed with the AMD root key hierarchy. The report contains a `measurement` field—the SHA-384 hash of the initial VM image and configuration—equivalent to SGX's MRENCLAVE but at VM granularity.

A critical difference from SGX is that SEV-SNP attestation can be requested at any time during VM execution (not just at initialization), enabling *dynamic attestation* for long-running workloads. The PaperClip system leverages this to support periodic re-attestation of active nodes.

5.4 Trust Chain and Certification Hierarchy

The complete trust chain for PaperClip attestation is:

$$\underbrace{\text{Intel/AMD Root CA}}_{\text{Vendor PKI}} \rightarrow \underbrace{\text{PCK / Chip ID Cert}}_{\text{Per-processor}} \rightarrow \underbrace{\text{Attestation Key}}_{\text{Quote signing key}} \rightarrow \underbrace{\text{Quote}}_{\text{Per-attestation}} \rightarrow \underbrace{\text{Enclave PubKey}}_{\text{Per-enclave}}$$

Each arrow represents a cryptographic binding, anchoring the enclave's public key (used for workload encryption) all the way back to the processor manufacturer's root of trust. This chain means that trusting the enclave's public key requires only trusting the chip manufacturer—a trust assumption that is standard in hardware security contexts and considerably narrower than trusting a cloud provider's software stack.

6 Cryptographic Model

6.1 Formal Security Goals

We formalize PaperClip's cryptographic goals as follows:

1. **Workload Confidentiality (WC):** For any computationally bounded adversary $\mathcal{A}$ who observes all network traffic and controls the host OS, the probability of $\mathcal{A}$ learning any function of the workload plaintext is negligible.

2. **Result Confidentiality (RC):** For any computationally bounded $\mathcal{A}$ who observes all network traffic, the probability of $\mathcal{A}$ learning any function of the result plaintext is negligible.

3. **Key Authenticity (KA):** Any workload encrypted under an attested public key pk_e will be decrypted by, and only by, the enclave whose measurement appears in the attestation report containing pk_e.

4. **Result Integrity (RI):** The decrypted result reflects the output of the correct workload executed on the correct input, rather than a fabricated or modified value.

6.2 Key Establishment Protocol

The PaperClip key establishment protocol is a variant of ECIES (Elliptic Curve Integrated Encryption Scheme) bootstrapped by remote attestation. Let the following notation hold:

- (sk_e, pk_e): Enclave key pair, generated inside the TEE at initialization; sk_e never exits the enclave boundary
- (sk_u, pk_u): User ephemeral key pair, generated in the browser for each job
- n: Fresh nonce issued by the API server, bound to job ID j
- W: Plaintext workload
- R: Plaintext result
- $\texttt{ECDH}(a, b)$: Elliptic Curve Diffie-Hellman with keys a, b
- $\texttt{HKDF}(k, \text{info})$: HMAC-based Key Derivation Function
- $\texttt{AES-GCM}(k, m)$: AES-256-GCM authenticated encryption

Workload Encryption (Client → Enclave):

$$(sk_e, pk_e) \leftarrow \text{KeyGen}() \quad \text{[inside enclave]} \tag{1}$$

$$pk_e \rightarrow \text{QuoteReportData} \rightarrow \text{Attestation} \tag{2}$$

$$(sk_u, pk_u) \leftarrow \text{KeyGen}() \quad \text{[client-side]} \tag{3}$$

$$Z = \texttt{ECDH}(sk_u, pk_e) \tag{4}$$

$$k_{enc} = \texttt{HKDF}(Z, \text{"paperclip-workload"} \| n) \tag{5}$$

$$C_W = \texttt{AES-GCM}(k_{enc}, W \| pk_u \| n) \tag{6}$$

$$\text{Submission} = (pk_u, C_W, j) \tag{7}$$

Workload Decryption (Inside Enclave):

$$Z' = \texttt{ECDH}(sk_e, pk_u) \tag{8}$$

$$k'_{enc} = \texttt{HKDF}(Z', \text{"paperclip-workload"} \| n) \tag{9}$$

$$W' = \texttt{AES-GCM-Dec}(k'_{enc}, C_W) \tag{10}$$

By the security of ECDH and HKDF, $k_{enc} = k'_{enc}$ only when sk_e and pk_u are the correct counterparts—i.e., only when both the correct enclave private key and the correct ephemeral user public key are used.

Result Encryption (Enclave → Client):

After execution, the enclave encrypts result R under pk_u extracted from W':

$$r_{eph} \leftarrow \text{KeyGen}() \quad \text{[ephemeral, inside enclave]} \tag{11}$$

$$Z_r = \texttt{ECDH}(r_{eph}, pk_u) \tag{12}$$

$$k_r = \texttt{HKDF}(Z_r, \text{"paperclip-result"} \| j) \tag{13}$$

$$C_R = \texttt{AES-GCM}(k_r, R \| j) \tag{14}$$

The client decrypts C_R using sk_u (the ephemeral private key stored in `sessionStorage`).

6.3 Security Properties

WC is achieved because C_W is semantically secure under AES-256-GCM and k_{enc} is computationally indistinguishable from random for any party who does not know both sk_e and sk_u. The adversary observing network traffic sees only (pk_u, C_W) and cannot derive k_{enc} without sk_e, which exists only inside the enclave.

RC is achieved by the same argument applied to result encryption, with sk_u playing the role of the secret.

KA is achieved by the attestation chain: the user verifies that pk_e is bound to the enclave measurement MRENCLAVE via the attestation quote, which is signed by the processor's attestation key in the Intel PKI. Encrypting under pk_e therefore guarantees decryption by the correctly measured enclave.

RI relies on the TEE's integrity guarantees: because MRENCLAVE measures the complete enclave code, the attested enclave will execute the workload correctly according to its specification. A compromised host cannot modify the enclave code without changing MRENCLAVE and thus failing attestation.

6.4 Nonce-Based Replay Prevention

The job nonce n plays a critical role in preventing replay attacks. Without it, an adversary who captures a valid C_W submission could re-submit the same ciphertext to execute the same workload again, potentially for billing fraud or to probe timing side-channels. The nonce n is generated server-side, bound to job ID j in the database, and marked as consumed after first use. The enclave validates n against a server-provided expected value before executing the workload.

7 Threat Model

7.1 Adversary Classification

We define four adversary classes relevant to PaperClip:

7.1.1 $\mathcal{A}_1$: Malicious Infrastructure Provider

This is the primary adversary that PaperClip is designed to defeat. $\mathcal{A}_1$ operates the physical hardware and has full control of:

- The host operating system, including kernel, device drivers, and system calls
- The hypervisor (if the node runs in a virtualized environment)
- The BIOS/UEFI firmware (in the CPU-only attacker model, not the hardware attacker model)
- All network interfaces on the host
- The storage subsystem (disk, memory-mapped I/O)

Despite this elevated privilege, $\mathcal{A}_1$ cannot access enclave memory contents (protected by hardware memory encryption), cannot produce valid attestation reports for modified enclave code (as the processor measures code at load time), and cannot forge attestation quotes (as the processor's signing key is fused in hardware and never accessible to software).

7.1.2 $\mathcal{A}_2$: Network Attacker

$\mathcal{A}_2$ can observe, inject, replay, and modify arbitrary network traffic between all parties. This model includes:

- Passive eavesdropping on all network communications
- Active manipulation of TCP/IP packets
- TLS interception via certificate spoofing (assuming a compromised CA)
- IP-level traffic analysis

Against $\mathcal{A}_2$, PaperClip provides workload confidentiality through end-to-end encryption (workloads are encrypted under the enclave key before leaving the client), and result confidentiality through the result encryption protocol. TLS provides protection against passive eavesdropping on metadata. The nonce prevents replay attacks.

7.1.3 $\mathcal{A}_3$: Compromised API Gateway

$\mathcal{A}_3$ represents a compromised or malicious API gateway operator. This adversary can:

- Read all metadata (job IDs, timestamps, user identifiers, node selections)
- Observe encrypted workload ciphertext
- Withhold or delay job submissions
- Modify job routing to direct workloads to malicious nodes

PaperClip mitigates $\mathcal{A}_3$ through end-to-end workload encryption (the gateway sees only ciphertext) and attestation pinning (the client verifies the attestation of the target node independently, so malicious node substitution is detectable).

7.1.4 $\mathcal{A}_4$: Hardware-Level Attacker

$\mathcal{A}_4$ can perform physical attacks on the hardware:

- Cold boot attacks to extract DRAM contents
- Bus snooping on PCIe or DDR4/DDR5 buses
- Power analysis and electromagnetic side-channel attacks
- Fault injection (voltage, clock glitching, laser fault injection)

Current TEE designs provide *limited* protection against $\mathcal{A}_4$. Intel SGX's Memory Encryption Engine protects against DRAM bus snooping via AES-XTS. However, sophisticated physical attacks such as Plundervolt [25] (voltage glitching to induce SGX faults) and DRAMbleed [29] (DRAM side-channel) have demonstrated exploitation of SGX. AMD SEV memory encryption similarly relies on on-die key storage, which is robust against bus snooping but may be vulnerable to advanced invasive attacks.

PaperClip explicitly scopes its confidentiality guarantee to software-level attackers ($\mathcal{A}_1$, $\mathcal{A}_2$, $\mathcal{A}_3$), noting that hardware-level attacks require physical access and significant resources, placing them outside the threat model of most practical compute marketplace deployments.

7.2 Out-of-Scope Threats

The following are explicitly outside PaperClip's current threat model:

- **Side-channel attacks:** Cache-timing attacks [27], speculative execution leakage (Spectre/Meltdown [20]), and page fault side-channels [28] can leak information from SGX enclaves. PaperClip does not mitigate these at the protocol level, though workloads can be written to be side-channel-resistant.
- **Software vulnerabilities in the enclave:** A buggy PaperClip node agent with a memory corruption vulnerability could be exploited to exfiltrate data. Enclave code minimization and formal verification are mitigations for future work.
- **Denial-of-service:** A provider can refuse to execute jobs or return garbage. PaperClip's reputation system partially mitigates this economically but does not cryptographically prevent it.

8 Security Analysis

8.1 Confidentiality Analysis

We argue workload confidentiality WC against adversary $\mathcal{A}_1 \cup \mathcal{A}_2 \cup \mathcal{A}_3$.

Against $\mathcal{A}_1$ (malicious infrastructure provider): Workload ciphertext C_W is transmitted from the client to the API gateway over TLS and then forwarded to the Node Agent over an authenticated internal channel. Even if $\mathcal{A}_1$ controls both the API gateway and the Node Agent at the OS level, the only information $\mathcal{A}_1$ observes is (pk_u, C_W, j). Without sk_e, $\mathcal{A}_1$ cannot derive k_{enc} and therefore cannot decrypt C_W. The key sk_e exists only inside the SGX EPC, which is hardware-encrypted; the OS cannot read EPC contents. Therefore WC holds against $\mathcal{A}_1$.

Against $\mathcal{A}_2$ (network attacker): All client-to-API-gateway traffic is protected by TLS 1.3. Even without TLS, the workload is encrypted end-to-end. The adversary observes ciphertext and cannot decrypt without sk_e.

Against $\mathcal{A}_3$ (compromised gateway): The gateway operator observes only (pk_u, C_W, j). As argued above, C_W is semantically secure without sk_e.

8.2 Integrity Analysis

Result integrity RI requires that the decrypted result R corresponds to the correct execution of workload W. This holds because:

1. Attestation verifies MRENCLAVE, guaranteeing the enclave runs the correct PaperClip node agent code.
2. The node agent code, upon receiving C_W, (a) decrypts it with sk_e, (b) verifies the nonce n, and (c) executes the workload in the sandbox and returns the result. The code cannot be modified without changing MRENCLAVE.
3. The result is encrypted by the enclave under pk_u. The enclave code that produces and encrypts the result is the same code that was measured by MRENCLAVE. Therefore, if MRENCLAVE verification passed, the result was produced by the correct code.

8.3 Authenticity of Enclave Identity

KA holds because:

1. The Intel attestation service cryptographically verifies that the Quote was produced by a genuine Intel SGX processor.

2. The MRENCLAVE in the Quote matches the expected value for the PaperClip node agent.

3. The pk_e hash is embedded in the Quote's REPORTDATA field, binding the enclave's public key to its measured identity.

4. Therefore, encrypting under pk_e guarantees decryption by a genuine enclave running the correct code.

9 GPU Confidential Computing

9.1 The GPU Privacy Gap

Traditional CPU-based TEEs (SGX, SEV-SNP) protect workloads during CPU computation but do not protect data transferred to GPU memory for accelerated computation. A standard deep learning training job that submits tensors to an NVIDIA GPU leaves those tensors in plaintext in GPU VRAM—directly readable by a sufficiently privileged host process. This is the GPU privacy gap, and it is critical for AI workloads because the GPU is where the computation—and the sensitive data—actually lives.

9.2 NVIDIA H100 Confidential Computing

The NVIDIA H100 Hopper architecture introduced hardware support for confidential computing through three mechanisms [5]:

9.2.1 Hardware Memory Encryption

H100's memory controller includes an inline AES-XTS-256 encryption engine that encrypts all data in HBM3 memory. Encryption keys are generated by the H100's on-die security processor and are not accessible to the CPU-side software stack (including the hypervisor). Data is decrypted transparently as the GPU executes CUDA kernels.

9.2.2 GPU Attestation

H100 implements a GPU attestation service (NVIDIA RIM Service) that allows a relying party to verify the GPU's identity and firmware state. The attestation report includes the GPU's hardware root certificate (GPC certificate, signed by NVIDIA's root CA), the current firmware version, and a measurement of the loaded CUDA driver stack. This attestation is integrated into PaperClip's node attestation workflow: nodes with H100 GPUs provide a combined CPU+GPU attestation report.

9.2.3 Confidential Compute Mode

When a VM is launched in confidential compute mode with an H100, the CUDA driver establishes an encrypted channel between the CPU-side guest driver and the GPU's firmware. DMA transfers to and from the GPU are encrypted in transit using keys negotiated via the CPU-GPU TLS-like handshake. This prevents the hypervisor from observing tensor data during PCIe transfers.

9.3 Integration with PaperClip

Nodes equipped with H100 GPUs register as a specialized node type with the flag `gpu_confidential: true`. Their attestation record includes the GPU's RIM attestation alongside the CPU TEE attestation. Users filtering for AI workloads requiring GPU can select these nodes and receive the combined attestation guarantee.

The execution environment on an H100-equipped node uses PyTorch or JAX with the CUDA confidential compute driver, running inside an AMD SEV-SNP protected VM. This provides a complete confidential AI execution stack:

1. AMD SEV-SNP protects the guest VM from the hypervisor, including the PyTorch process and model weights in CPU RAM.

2. The H100 confidential compute mode protects the PCIe DMA transfers and GPU HBM memory from the hypervisor.

3. The combined CPU+GPU attestation allows the user to verify both protection layers before submitting.

9.4 Limitations of Current GPU Confidential Computing

Despite H100's advances, several limitations remain relevant to PaperClip:

- **PCIe bandwidth overhead:** Encryption of DMA transfers introduces approximately 5–12% latency overhead for data-intensive workloads [5].
- **No multi-GPU confidential attestation:** H100 attestation is per-card; multi-card NVLink clusters do not yet have a unified attestation model.
- **CUDA kernel measurement:** Unlike SGX's fine-grained code measurement, H100 attestation measures only the firmware/driver stack, not the user's CUDA kernels themselves. A malicious provider could theoretically run different kernels than claimed.
- **Memory management:** CUDA's unified memory model, which pages memory between CPU RAM and GPU HBM, must be configured carefully to ensure that paged-out GPU data is protected by the CPU TEE when resident in CPU RAM.

10 Distributed Job Scheduler

10.1 Scheduler Architecture

The PaperClip scheduler is a centralized service (with replication for fault tolerance) implementing a multi-criteria best-fit algorithm. It maintains:

- A **Job Queue:** A priority queue of pending job submissions, ordered by user-specified priority and submission time.
- A **Node Table:** An in-memory replica of the node registry, refreshed periodically, containing each node's current capacity, price, attestation status, and estimated queue depth.
- A **Scoring Engine:** A weighted multi-criteria scoring function that evaluates each candidate node for a given job.

10.2 Multi-Criteria Node Scoring

For job j and candidate node n, the scheduler computes a composite score:

$$S(j,n) = w_1 \cdot F_{price}(j,n) + w_2 \cdot F_{latency}(j,n) + w_3 \cdot F_{rep}(n) + w_4 \cdot F_{tee}(j,n) + w_5 \cdot F_{cap}(j,n) \tag{15}$$

where:

- $F_{price}(j,n) = 1 - \frac{p_n - p_{min}}{p_{max} - p_{min}}$: Normalized price factor (lower price → higher score)
- $F_{latency}(j,n) = 1 - \frac{d_{n,\text{user}} - d_{min}}{d_{max} - d_{min}}$: Normalized network latency from node to user's region
- $F_{rep}(n) = \frac{\text{completed}(n)}{\text{submitted}(n)} \cdot \text{avg_rating}(n)/5$: Reputation score combining completion rate and user ratings
- $F_{tee}(j,n) \in \{0, 0.5, 1\}$: TEE compatibility score (1 = required TEE type available, 0.5 = equivalent TEE, 0 = incompatible)
- $F_{cap}(j,n) \in \{0, 1\}$: Capability check (0 if node cannot satisfy job resource requirements, 1 otherwise)

Default weights $(w_1, w_2, w_3, w_4, w_5) = (0.30, 0.25, 0.20, 0.15, 0.10)$ reflect a user preference ordering that prioritizes price and latency while maintaining quality and security constraints. Users can override weights per job submission.

Algorithm 1 PaperClip Job Scheduling Algorithm

Require: Job queue Q, Node table N, Score weights $\mathbf{w}$
Ensure: Job assignments A

```
while Q is not empty do
    j ← dequeue_highest_priority(Q)
    candidates ← {n ∈ N : n.isActive ∧ n.attestationValid ∧ F_cap(j,n) = 1}
    if candidates = ∅ then
        requeue(j, delay=30s)
        continue
    end if
    scores ← {S(j,n) : n ∈ candidates}
    n* ← arg max_{n∈candidates} S(j,n)
    if S(j,n*) < θ_min then
        notify_user(j, msg="No suitable nodes, waiting...")
        requeue(j, delay=60s)
        continue
    end if
    success ← dispatch(j, n*)
    if success then
        A ← A ∪ {(j,n*)}
        n*.currentLoad ← n*.currentLoad + j.estimatedLoad
    else
        mark_unavailable(n*, duration=120s)
        requeue_front(j)
    end if
end while
```

The scheduler implementation on the backend:

```
// server/services/SchedulerService.js
const EventEmitter = require('events');
```

```
const NodeService  = require('./NodeService');
const JobService   = require('./JobService');

class SchedulerService extends EventEmitter {
  constructor() {
    super();
    this.running        = false;
    this.loopInterval   = null;
    this.MIN_SCORE      = 0.25;
    this.WEIGHTS = {
      price:   0.30,
      latency: 0.25,
      rep:     0.20,
      tee:     0.15,
      cap:     0.10
    };
  }

  start() {
    this.running = true;
    this.loopInterval = setInterval(
      () => this.schedulerLoop(), 2000
    );
    console.log('Scheduler started');
  }

  async schedulerLoop() {
    try {
      const pendingJobs = await JobService
        .getPendingJobs({ limit: 50 });
      if (pendingJobs.length === 0) return;

      const activeNodes = await NodeService
        .getActiveAttestedNodes();

      for (const job of pendingJobs) {
        await this.scheduleJob(job, activeNodes);
      }
    } catch (err) {
      console.error('Scheduler loop error:', err);
    }
  }

  async scheduleJob(job, nodes) {
    // Filter to capability-compatible nodes only
    const candidates = nodes.filter(n =>
      this.checkCapabilityCompatibility(job, n)
    );
    if (candidates.length === 0) {
      await JobService.requeueJob(job.id, 30000);
      return;
    }

    // Score all candidates
    const scored = candidates.map(n => ({
      node:  n,
      score: this.scoreNode(job, n, candidates)
    })).sort((a, b) => b.score - a.score);

    const best = scored[0];
    if (best.score < this.MIN_SCORE) {
      await JobService.requeueJob(job.id, 60000);
      return;
    }

    // Attempt dispatch
    try {
      await JobService.assignJobToNode(
        job.id, best.node.id
      );
      await NodeService.dispatchJob(
        best.node.id, job
      );
      this.emit('job:dispatched', {
        jobId: job.id,
        nodeId: best.node.id,
        score: best.score
      });
    } catch (err) {
      // Node failed to accept job temporarily penalize
      await NodeService.markTemporarilyUnavailable(
        best.node.id, 120000
      );
      await JobService.requeueJobFront(job.id);
    }
  }

  scoreNode(job, node, allCandidates) {
    const prices   = allCandidates.map(n => n.pricePerHour);
    const latencies= allCandidates.map(n =>
      n.latencyToRegion[job.userRegion] || 999
    );
    const pMin = Math.min(...prices);
    const pMax = Math.max(...prices);
    const lMin = Math.min(...latencies);
    const lMax = Math.max(...latencies);

    const fPrice = pMax === pMin ? 1 :
      1 - (node.pricePerHour - pMin) / (pMax - pMin);

    const nodeLat = node.latencyToRegion[
      job.userRegion
    ] || 999;
    const fLatency = lMax === lMin ? 1 :
      1 - (nodeLat - lMin) / (lMax - lMin);

    const fRep = (node.completionRate || 0.5) *
                 ((node.avgRating || 3) / 5);

    const fTee = node.teeType === job.requiredTeeType
      ? 1.0
      : (this.isTeeEquivalent(node.teeType,
                              job.requiredTeeType)
         ? 0.5 : 0);

    if (fTee === 0) return 0; // Hard
```

```
    incompatibility

    return (
      this.WEIGHTS.price    * fPrice +
      this.WEIGHTS.latency  * fLatency +
      this.WEIGHTS.rep      * fRep +
      this.WEIGHTS.tee      * fTee +
      this.WEIGHTS.cap      * 1.0  // Already filtered
    );
  }

  checkCapabilityCompatibility(job, node) {
    if (job.requiresGpu && !node.hasGpu) return false;
    if (job.minMemoryGB > node.availMemGB) return false;
    if (job.minEpcGB > (node.availEpcGB||0)) return false;
    if (job.maxDuration > node.maxJobDuration) return false;
    return node.attestationStatus === 'VERIFIED';
  }

  isTeeEquivalent(have, want) {
    const equivalents = [['SGX_V1','SGX_V2']];
    return equivalents.some(g =>
      g.includes(have) && g.includes(want)
    );
  }
}

module.exports = new SchedulerService();
```

Listing 11: Core scheduler dispatch loop implementing multi-criteria node selection with failover and requeue logic.

10.3 Queue Management and Fairness

The job queue is implemented as a multi-level priority queue with three levels: `URGENT` (SLA-bound workloads with time deadlines), `NORMAL` (standard marketplace submissions), and `BATCH` (low-priority batch jobs). Within each level, jobs are ordered by a combination of submission time and user reputation score, preventing starvation of low-priority jobs through an aging mechanism that gradually increases a job's effective priority as its wait time grows.

10.4 Node Load Balancing

The scheduler tracks estimated load on each node as a dimensionless utilization metric combining CPU utilization, memory utilization, and EPC utilization. The node scoring function naturally provides soft load balancing through the latency component: a heavily loaded node will exhibit higher processing latency for queued jobs, reducing its effective latency score. For large-scale deployments, the scheduler can be sharded by geographic region, with a cross-region scheduler handling workloads requiring inter-region routing.

11 Economic Marketplace Model

11.1 Market Structure

PaperClip operates as a two-sided marketplace with compute providers on the supply side and users with AI workloads on the demand side. The marketplace operator (PaperClip) facilitates matching, attestation, and payment settlement, charging a platform fee of δ (e.g., 10%) on each completed transaction.

11.2 Provider Incentive Design

Compute providers are economic agents who invest in hardware and operating infrastructure. A provider with hardware cost H, operating cost per hour $C_{ops}(H)$ (dominated by electricity cost), and marketplace revenue per hour R will participate when:

$$R > C_{ops}(H) + \frac{H}{T_{amortize}} \tag{16}$$

where $T_{amortize}$ is the amortization period (typically 24–36 months for server hardware). The key insight is that $C_{ops}(H)$ is dominated by electricity cost, and electricity prices vary dramatically by geography. In regions with hydroelectric or renewable energy abundance (e.g., Iceland, Quebec, Pacific Northwest USA, Paraguay), electricity costs can be as low as \$0.03–0.05/kWh, compared to \$0.12–0.25/kWh in data center hubs on the US East Coast. This creates strong economic incentives for providers in low-cost energy regions to offer competitive confidential compute capacity.

For a single H100 GPU drawing 700W average under load, the electricity cost differential between a low-cost region (\$0.04/kWh) and a typical US data center (\$0.16/kWh) is:

$$\Delta C_{elec} = 0.7 \text{ kW} \times (\$0.16 - \$0.04)/\text{kWh} \times 24 \text{ h/day} \tag{17}$$

$$= \$2.02/\text{GPU/day} \tag{18}$$

At scale, this energy arbitrage significantly shapes the geographic distribution of PaperClip providers.

11.3 Pricing Mechanisms

PaperClip supports three pricing mechanisms:

Posted Prices: The provider specifies a fixed price per compute unit (e.g., per GPU-hour, per SGX-second). Users see a deterministic cost before submission. This is the default mechanism for small to medium workloads.

Spot Pricing: Providers can expose capacity at variable spot prices that adjust based on current demand. The scheduler weights spot prices against the baseline score function. Spot pricing enables price discovery and market efficiency for large, flexible workloads.

Reserve Auctions: For large, long-running workloads (multi-hour training jobs), users can submit bids specifying their maximum willingness-to-pay. The scheduler runs a second-price auction among available nodes whose capacity satisfies the job requirements.

11.4 Reputation and Quality Enforcement

Provider reputation is critical for marketplace health. The PaperClip reputation system tracks:

- **Job Completion Rate:** Fraction of dispatched jobs completed successfully, penalizing providers who crash, reject, or time out jobs.
- **Attestation Freshness:** How frequently the provider re-attests their nodes. Stale attestations reduce reputation.
- **Execution Time Accuracy:** How closely actual execution times match the provider's advertised estimates.
- **User Ratings:** Explicit satisfaction ratings from users after job completion.

Providers with reputation below a minimum threshold ρ_{min} are delisted from the marketplace. This Sybil-resistance is reinforced by requiring a staking deposit from new providers, which is slashed for malfeasance.

11.5 Economic Security Against Manipulation

A collusion attack where a provider offers cheap prices to win many jobs and then fails to execute them is mitigated by: (1) the staking/slashing mechanism makes defection economically costly, (2) the reputation system rapidly reduces the defecting provider's score, and (3) the payment settlement escrows user funds and releases them only upon verified job completion (with the enclave's signed execution receipt serving as proof of completion).

12 Prototype Implementation

12.1 Implementation Overview

The PaperClip prototype is implemented as a full-stack web application with the following technology choices, each motivated by specific technical requirements:

- **Frontend:** React.js 18 with React Router 6, using the Context API for state management and the jose library for JWE encryption/decryption. React was chosen for its component model, which maps cleanly to the marketplace's distinct functional areas (node browsing, job submission, result viewing).
- **API Backend:** Node.js 20 with Express 4.18, providing the REST API. Node.js's asynchronous I/O model is well-suited for the API gateway's role as an orchestrator that awaits responses from multiple downstream services.
- **Database:** PostgreSQL 16 for persistent storage of nodes, jobs, users, and attestation records. Redis 7 for session storage, job queue state, and frequently-accessed node metadata cache.
- **Attestation Integration:** REST client calling Intel Trust Authority (ITA) for SGX quote verification, with AMD Attestation Service integration for SEV-SNP nodes.
- **Enclave Prototype:** A Node.js-based mock enclave that simulates the SGX execution environment for testing, with a Gramine LibOS-based real enclave for production validation.

12.2 Dashboard Implementation

The user dashboard provides an overview of active jobs, recent completions, spending, and account status:

```
// src/pages/Dashboard.jsx
import React, { useState, useEffect } from 'react';
import { useAuth } from '../context/AuthContext';
import { jobsApi, nodesApi } from '../api/client';
```

```
import JobStatusCard from '../components/JobStatusCard';
import SpendingChart  from '../components/SpendingChart';
import ActiveJobsTable from '../components/ActiveJobsTable';

export default function Dashboard() {
  const { user }                        = useAuth();
  const [stats,       setStats]         = useState(null);
  const [activeJobs, setActiveJobs]     = useState([]);
  const [recentJobs, setRecentJobs]     = useState([]);
  const [spending,    setSpending]      = useState([]);
  const [loading,     setLoading]       = useState(true);

  useEffect(() => {
    Promise.all([
      jobsApi.listByUser(user.id, {
        status: 'RUNNING', limit: 10
      }),
      jobsApi.listByUser(user.id, {
        status: 'COMPLETE', limit: 5
      }),
      jobsApi.getSpendingHistory(user.id, { days: 30 })
    ]).then(([active, recent, spend]) => {
      setActiveJobs(active.data.jobs);
      setRecentJobs(recent.data.jobs);
      setSpending(spend.data.dailySpend);
      setStats({
        totalJobs:    active.data.total + recent.data.total,
        activeCount:  active.data.total,
        totalSpend30d: spend.data.totalSpend,
        avgJobDuration: spend.data.avgDurationSeconds
      });
    }).finally(() => setLoading(false));
  }, [user.id]);

  return (
    <div className="dashboard">
      <h1>Welcome, {user.displayName}</h1>

      {/* Summary stats row */}
      {stats && (
        <div className="stats-row">
          <StatCard label="Active Jobs"
            value={stats.activeCount} />
          <StatCard label="Total Jobs"
            value={stats.totalJobs} />
          <StatCard label="Spend (30d)"
            value={`$${stats.totalSpend30d.toFixed(2)}`} />
          <StatCard label="Avg Duration"
            value={`${stats.avgJobDuration}s`} />
        </div>
      )}

      {/* Active jobs table */}
      <section>
        <h2>Active Jobs</h2>
        <ActiveJobsTable jobs={activeJobs} />
      </section>

      {/* 30-day spending chart */}
      <section>
        <h2>Spending (Last 30 Days)</h2>
        <SpendingChart data={spending} />
      </section>

      {/* Recent completions */}
      <section>
        <h2>Recent Completions</h2>
        {recentJobs.map(job => (
          <JobStatusCard key={job.id} job={job} />
        ))}
      </section>
    </div>
  );
}
```

Listing 12: Dashboard component aggregating job history, spending, and active job status.

12.3 Provider Registration Flow

Compute providers register through a dedicated onboarding flow that collects hardware specifications, submits an initial attestation quote, and sets pricing:

```
// server/routes/providers.js
const router = require('express').Router();
const { authenticate, requireRole } =
  require('../middleware/auth');
const AttestationService =
  require('../services/AttestationService');
const NodeService = require('../services/NodeService');

/**
 * POST /api/v1/providers/:providerId/nodes
 * Register a new compute node. Requires a valid
 * attestation quote to be admitted to the marketplace.
 */
router.post('/:providerId/nodes',
  authenticate,
  requireRole('provider'),
  async (req, res) => {
    try {
      const {
        nodeName, region, teeType, teeVersion,
        attestationQuote,  // Raw hex quote from node agent
        enclavePublicKey,  // PEM from quote REPORTDATA
        cpuModel, gpuModel, memGB, epcGB,
        pricePerHour, maxJobDurationSeconds,
        supportedWorkloadTypes, networkEndpoint
      } = req.body;
```

```

    // Validate required fields
    if (!attestationQuote || !enclavePublicKey) {
      return res.status(400).json({
        message: 'attestationQuote and ' +
          'enclavePublicKey are required'
      });
    }

    // Verify attestation before admitting node
    let verificationResult;
    if (teeType === 'SGX') {
      verificationResult = await AttestationService
        .verifySSGXQuote(req.params.providerId,
                         attestationQuote);
    } else if (teeType === 'SEV_SNP') {
      verificationResult = await AttestationService
        .verifySEVSNPReport(req.params.providerId,

    attestationQuote);
    } else {
      return res.status(400).json({
        message: `Unsupported TEE type: ${teeType}`
      });
    }

    if (verificationResult.status !== 'VERIFIED') {
      return res.status(403).json({
        message: 'Node attestation failed',
        detail: verificationResult.reason
      });
    }

    // Verify that enclavePublicKey matches
    // the hash in the quote REPORTDATA
    const pubKeyValid = await AttestationService
      .verifyPublicKeyBinding(
        attestationQuote,
        enclavePublicKey
      );
    if (!pubKeyValid) {
      return res.status(403).json({
        message: 'Enclave public key does not match' +
          ' quote REPORTDATA'
      });
    }

    // Create node record
    const node = await NodeService.createNode({
      providerId:     req.params.providerId,
      name:           nodeName,
      region,
      teeType,        teeVersion,
      enclavePublicKey,
      cpuModel,       gpuModel,
      memGB,          epcGB,
      pricePerHour,
    maxJobDurationSeconds,
      supportedWorkloadTypes,
      networkEndpoint,
      attestationRecord: verificationResult,
      isActive: true
    });

    res.status(201).json({ node });
  } catch (err) {
    res.status(500).json({
      message: 'Node registration failed',
      error: err.message
    });
  }
}
);

module.exports = router;
```

Listing 13: Provider node registration handler on the server side, integrating attestation verification before node admission.

This registration handler enforces a critical security invariant: no node is admitted to the marketplace without passing attestation verification. The double check—verifying the attestation quote independently of the provider's claim, and verifying the public key binding—ensures that even a compromised provider registration flow cannot admit an unattestatd node.

12.4 Database Schema

The PaperClip data model is organized around six core entities. The jobs table captures the full lifecycle of each workload submission:

```
-- PostgreSQL schema: jobs table
CREATE TABLE jobs (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id         UUID NOT NULL REFERENCES users(id),
    node_id         UUID REFERENCES nodes(id),

    -- Status lifecycle
    status          VARCHAR(32) NOT NULL DEFAULT 'PENDING',
                    -- PENDING|QUEUED|DISPATCHED|RUNNING
                    -- |COMPLETE|FAILED|CANCELLED

    -- Job configuration
    workload_type   VARCHAR(64)  NOT NULL,
    required_tee    VARCHAR(32),
    -- SGX|SEV_SNP|ANY
```

```sql
    requires_gpu     BOOLEAN       DEFAULT FALSE,
    min_memory_gb    FLOAT,
    max_duration_s   INTEGER,
    user_region      VARCHAR(32),
    -- for latency scoring

    -- Cryptographic fields
    -- nonce is bound to this job ID in key derivation
    nonce            VARCHAR(128) NOT NULL UNIQUE,
    user_pubkey_jwk JSONB,
    -- ephemeral pub key
    -- Only ciphertext is stored server-side;
    -- private key never leaves client
    encrypted_payload TEXT,
    encrypted_result  TEXT,
    result_hash       VARCHAR(128),
    -- HMAC of result

    -- Economic fields
    price_per_hour   DECIMAL(10,6),
    actual_cost      DECIMAL(10,6),

    -- Timing
    submitted_at     TIMESTAMPTZ DEFAULT NOW(),
    dispatched_at    TIMESTAMPTZ,
    started_at       TIMESTAMPTZ,
    completed_at     TIMESTAMPTZ,

    -- Execution metadata
    error_message    TEXT,
    execution_time_s INTEGER,

    CONSTRAINT valid_status CHECK (
      status IN (
        'PENDING','QUEUED','DISPATCHED','RUNNING',
        'COMPLETE','FAILED','CANCELLED'
      )
    )
);

CREATE INDEX idx_jobs_user_status
  ON jobs(user_id, status);
CREATE INDEX idx_jobs_node_status
  ON jobs(node_id, status);
CREATE INDEX idx_jobs_pending
  ON jobs(status, submitted_at)
  WHERE status = 'PENDING';
```

Listing 14: Core database schema for the jobs table, capturing the full job lifecycle from submission to result retrieval.

The schema design reflects the security architecture: the `encrypted_payload` and `encrypted_result` columns store only ciphertext—the API server can persist and retrieve job data without ever accessing plaintext workload content. The `nonce` column has a UNIQUE constraint enforcing single-use at the database level as a defense-in-depth against replay attacks.

13 Evaluation Plan

13.1 Evaluation Goals

Our evaluation aims to quantify four key properties of the PaperClip system:

1. **TEE Overhead:** What is the performance cost of running workloads inside SGX enclaves or SEV-SNP VMs relative to native execution?
2. **Scheduler Efficiency:** How efficiently does the multi-criteria scheduler allocate jobs to nodes relative to optimal allocation?
3. **Attestation Latency:** How much end-to-end latency does the attestation workflow add relative to a non-attestation baseline?
4. **Marketplace Dynamics:** How do prices, latency, and throughput behave at varying marketplace load levels?

13.2 TEE Overhead Experiments

We propose the following experimental setup for measuring SGX enclave overhead:

Workloads: Three representative AI workloads spanning the compute profile of typical PaperClip jobs:

- **Microbenchmark:** Matrix multiplication ($N \times N$ for $N \in \{256, 512, 1024, 2048\}$) measuring raw compute throughput inside EPC.
- **Inference workload:** BERT-base inference on a batch of 32 inputs (768-dimensional embeddings, 12 layers), measuring end-to-end latency.
- **Training workload:** Fine-tuning a 110M parameter model on a 10k-sample dataset for 1 epoch, measuring training throughput (samples/second).

Execution environments:

1. Native (no enclave) — baseline
2. SGX enclave via Gramine LibOS
3. AMD SEV-SNP VM (QEMU/KVM)
4. SGX enclave with H100 GPU passthrough (confidential compute mode)

Metrics: Execution time (mean, P95, P99), memory overhead (EPC vs. non-EPC), context switch overhead (EPC EWB — enclave page eviction), GPU kernel latency.

Expected results: SGX v2 introduces approximately 5–15% overhead for memory-bound workloads due to EPC memory encryption. For compute-bound matrix multiply, the overhead is dominated by EPC paging if the working set exceeds available EPC; with Ice Lake's 512 MB EPC, BERT-base inference should fit without paging. SEV-SNP overhead is expected to be lower (2–8%) for these workloads due to VM-granularity rather than page-level isolation.

13.3 Attestation Latency Experiments

The attestation workflow adds latency to the node selection process. We measure:

- **Quote generation time:** Time for a Quoting Enclave to sign an EREPORT (expected: 50–200 ms)
- **ITA verification round-trip:** Time from quote submission to Intel Trust Authority to receiving a verification JWT (expected: 200–800 ms depending on ITA load)
- **Total attestation latency:** User-observed delay from clicking "Submit Job" to receiving the attested public key

We further evaluate the effect of attestation caching: since attestation reports are valid for a configurable freshness window (e.g., 60 minutes), a cached valid attestation eliminates the ITA verification step for returning users visiting the same node.

13.4 Scheduler Efficiency Experiments

We evaluate scheduler quality by comparing the multi-criteria scoring algorithm against: (1) random assignment, (2) price-only selection, and (3) an optimal offline assignment computed by integer linear programming (ILP) using the Gurobi solver.

The experiment generates synthetic workload traces sampled from realistic distributions: job inter-arrival times from an exponential distribution (mean 5 seconds), job durations from a log-normal distribution (mean 120 seconds, $\sigma = 1.5$), and node prices from a Pareto distribution reflecting market diversity. We measure:

- **Mean job wait time** from queue entry to dispatch
- **Price efficiency** (ratio of actual spend to optimal spend)
- **Node utilization variance** across the marketplace
- **Scheduling throughput** (jobs scheduled per second at scale)

13.5 Marketplace Load Experiments

We deploy a simulated marketplace with 100 provider nodes across 5 geographic regions and drive load at rates of 1, 10, 100, and 1000 job submissions per second. Metrics include API gateway throughput (requests/second), scheduler queue depth, job completion latency at P50/P95/P99, and marketplace throughput under failures (injecting 10% node failure rate).

14 Related Work

14.1 Trusted Execution Environments

Intel SGX [1] introduced hardware-level enclave isolation for x86 processors. Haven [14] demonstrated running unmodified Windows applications inside SGX enclaves using a shielding execution environment. Graphene [15] and its successor Gramine [16] provide library OS implementations for running Linux applications in SGX. VC3 [17] applied SGX to privacy-preserving MapReduce. Eleos [19] analyzed and reduced exit-less call overhead in SGX. OpenEnclave SDK [18] provides a vendor-neutral API for writing portable enclave applications.

AMD SEV [2] introduced VM-level memory encryption. SEV-SNP [3] extended SEV with integrity guarantees through the Reverse Map Table. Confidential VMs on AMD EPYC processors have been deployed in production by Google Cloud (Confidential VMs) and Azure (DCasv5-series).

ARM TrustZone [30] provides a dual-world isolation model widely used in mobile platforms. ARM CCA [4], introduced with ARMv9, adds Realm management for cloud-scale confidential workloads.

NVIDIA H100 [5] introduced confidential GPU computing. Telekine [40] previously demonstrated GPU encryption for SGX-protected workloads using software techniques before hardware support existed.

14.2 TEE Vulnerabilities

The security of SGX has been challenged by numerous side-channel attacks. Spectre and Meltdown [20, 21] demonstrated speculative execution leakage affecting SGX. Foreshadow (L1TF) [22] showed SGX attestation keys could be extracted via speculative execution. SGAxe [23] extended Foreshadow to break SGX attestation. CacheZoom [24] demonstrated cache side-channel attacks on SGX. Plundervolt [25] and Voltpillager [26] demonstrated SGX fault injection via voltage manipulation. Intel has responded with microcode mitigations and enhanced ISV attestation policies; PaperClip's node admission policy can enforce minimum TCB levels that exclude vulnerable processor configurations.

14.3 Privacy-Preserving Machine Learning

Cryptonets [7] demonstrated neural network inference on homomorphically encrypted data. GAZELLE [31] combined HE and MPC for efficient secure inference. SecureML [8] implemented privacy-preserving ML training using 2-party secure computation. DeepSecure [32] optimized garbled circuits for deep network inference. DELPHI [33] achieved practical secure inference performance using a hybrid HE/MPC/TEE approach.

FATE (Federated AI Technology Enabler) [39] provides a production federated learning framework. PySyft [37] combines federated learning with differential privacy. TensorFlow Federated [38] demonstrates large-scale federated learning at Google.

Slalom [13] pioneered the use of SGX for trusted ML inference, using the enclave to verify GPU computations through verifiable linear algebra. Our work generalizes this to a marketplace setting with arbitrary AI workloads.

14.4 Confidential Computing Platforms

Azure Confidential Computing [41] provides SGX-based VMs (DCsv3-series) and confidential Kubernetes node pools. AWS Nitro Enclaves [42] offer isolated enclave environments for EC2 instances, though without remote attestation to hardware roots of trust in the same sense as SGX. Google Cloud Confidential VMs [43] deploy AMD SEV-enabled C2D instances.

Fortanix Runtime Encryption [66] provides a commercial enclave runtime platform. Edgeless Systems [67] offers Marblerun, a confidential computing framework for containerized microservices. Anjuna [68] simplifies SGX and SEV-SNP deployment for enterprise applications.

14.5 Decentralized Compute Markets

Akash Network [44] provides a decentralized cloud compute marketplace using blockchain-based escrow and container-based workload execution, but without TEE-based confidentiality guarantees. Golem Network [45] offers decentralized compute sharing for CPU and GPU workloads, similarly without formal hardware attestation. Render Network [64] focuses on GPU rendering workloads in a decentralized setting.

iExec [46] integrates TEE-based confidential computing with blockchain payment for a decentralized oracle and compute marketplace, most closely related to PaperClip. PaperClip differentiates through its AI workload focus, multi-criteria scheduling, and integrated GPU confidential computing support.

Gensyn [47] proposes a decentralized ML training network with verification through probabilistic sampling rather than TEE-based confidentiality. Bittensor [48] creates economic incentives for distributed ML through tokenomics but without hardware privacy guarantees.

14.6 Attestation and Verification

TrustDump [65] proposed decentralized attestation for SGX using blockchain. Town Crier [56] implemented a trusted relay using SGX for blockchain-connected data feeds. Ekiden [57] built privacy-preserving smart contracts using SGX attestation. TEEMS [69] addressed revocation of TEE attestations in distributed systems.

Remote attestation for AMD SEV was studied in [58] and integrated with cloud orchestration in [59].

14.7 Distributed Systems and Scheduling

Borg [49] and Omega [50] describe Google's large-scale cluster management systems, which inform PaperClip's scheduler design. Mesos [51] introduced the two-level scheduling model. Kubernetes [52] has become the standard for containerized workload orchestration. Ray [53] provides distributed computing primitives for ML workloads.

Fair scheduling [54] and deadline-aware scheduling [55] are relevant to PaperClip's multi-level priority queue design.

15 Future Work

15.1 Confidential Multi-GPU Training

The most significant technical gap in the current PaperClip design is support for distributed multi-GPU training at the scale of large language model pre-training. Training a GPT-3 class model (175B parameters) requires hundreds of A100/H100 GPUs operating in parallel, with high-bandwidth NVLink and InfiniBand interconnects. Current H100 attestation is per-card and lacks a unified multi-card attestation model.

Future work should define a collective attestation protocol for NVLink-connected GPU clusters, where the attestation of the cluster as a whole binds to all constituent cards' identities. This requires hardware support for collective attestation report generation, analogous to how Intel SGX attestation quotes can report the complete enclave identity. NVIDIA's upcoming Blackwell architecture may introduce this capability.

15.2 Privacy-Preserving Fine-Tuning

PaperClip's current model treats workloads as black boxes. A more sophisticated integration would support privacy-preserving fine-tuning protocols, where the base model weights are encrypted under the enclave's key and the user's fine-tuning dataset is also encrypted, producing a fine-tuned model whose weights remain confidential to both the provider and the user (useful for model confidentiality as well as data confidentiality). This could leverage PEFT (Parameter-Efficient Fine-Tuning) techniques like LoRA inside the enclave to reduce the computation required within the limited EPC memory.

15.3 Decentralized Attestation Registry

The current PaperClip attestation service depends on Intel Trust Authority and AMD's attestation service, creating trust dependencies on those vendors. Future work could implement a decentralized attestation registry using a permissioned blockchain, where verification nodes collectively validate attestation quotes and record them on-chain, eliminating the need for a centralized attestation service. Light client protocols could allow users to verify attestation records without trusting any single node.

15.4 Confidential Federated Learning Orchestration

PaperClip could serve as the infrastructure layer for confidential federated learning, where the federated aggregation server runs inside an enclave, preventing the aggregation operator from observing individual model updates. This would address a key privacy limitation of federated learning that cannot be solved by FL alone.

15.5 Side-Channel Resistance

As discussed in Section 7, PaperClip does not currently address side-channel attacks on SGX. Future work should integrate side-channel-resistant execution libraries (e.g., using oblivious RAM (ORAM) for memory access pattern hiding [60]), constant-time cryptographic primitives, and hardware speculation barriers to harden the enclave against cache and timing attacks.

15.6 Formal Verification of the Enclave Runtime

The security argument for result integrity (RI) depends on the correctness of the enclave code. Future work should apply formal verification tools (e.g., Verified Software Toolchain [61], F* [62]) to formally verify the enclave's decryption, sandboxing, and re-encryption logic against a formal specification.

15.7 Cross-Platform TEE Abstraction

Currently, the PaperClip node agent requires separate implementations for SGX and SEV-SNP. A unified TEE abstraction layer—building on OpenEnclave SDK or the Enarx [63] project—would allow a single codebase to target multiple TEE backends, reducing implementation complexity and increasing the number of hardware platforms available in the marketplace.

16 Conclusion

This paper has presented PaperClip, a decentralized confidential compute marketplace for privacy-preserving AI workloads. We have described the

complete system architecture, including a five-tier distributed design spanning a React.js marketplace frontend, a Node.js/Express API gateway, a cryptographic attestation verification service, a multi-criteria distributed job scheduler, and confidential execution nodes equipped with Intel SGX, AMD SEV-SNP, and NVIDIA H100 confidential computing hardware.

The core security insight of PaperClip is that hardware-enforced TEE isolation, combined with remote attestation and end-to-end encrypted workload submission, allows users to delegate AI computation to untrusted third-party infrastructure while retaining cryptographic guarantees of workload confidentiality, result integrity, and enclave authenticity. The workload encryption protocol, based on ECIES with attestation-bound key establishment, ensures that plaintext data never travels outside the user's browser and the enclave's protected memory.

The multi-criteria scheduler integrates price, network latency, provider reputation, TEE type compatibility, and hardware capability into a composite scoring function that efficiently matches AI jobs to the best available confidential compute nodes. The economic marketplace model aligns provider incentives through posted prices, spot pricing, and reserve auctions, leveraging global electricity cost differentials to create a geographically distributed, competitive compute supply.

PaperClip represents a practical step toward a future in which sensitive AI workloads—clinical model fine-tuning, financial fraud detection, proprietary model inference—can be executed on commodity hardware operated by untrusted providers, without sacrificing the privacy guarantees that sensitive applications demand. As TEE hardware matures and GPU confidential computing becomes more capable, the PaperClip architecture provides a foundation for secure, decentralized AI infrastructure at global scale.

A Node Agent Enclave Runtime

The PaperClip node agent enclave runtime is responsible for receiving and decrypting job payloads, executing workloads in a sandboxed environment, and returning encrypted results. The enclave initialization sequence:

```c
/* enclave/paperclip_enclave.c */
#include "sgx_trts.h"
#include "sgx_tcrypto.h"
#include "sgx_report.h"
#include "openssl/evp.h"
#include "openssl/ec.h"

static EVP_PKEY *g_enclave_privkey  = NULL;
static uint8_t   g_enclave_pubkey_pem[512];
static size_t    g_enclave_pubkey_pem_len = 0;

/**
 * ecall_init_enclave: Called once at enclave startup.
 * Generates the enclave's ECDH-P384 key pair and
 * prepares the public key for inclusion in the
 * attestation report's REPORTDATA field.
 */
sgx_status_t ecall_init_enclave(void) {
    sgx_status_t ret = SGX_SUCCESS;

    /* Generate EC P-384 key pair inside enclave */
    EVP_PKEY_CTX *ctx = EVP_PKEY_CTX_new_id(
        EVP_PKEY_EC, NULL
    );
    EVP_PKEY_keygen_init(ctx);
    EVP_PKEY_CTX_set_ec_paramgen_curve_nid(
        ctx, NID_secp384r1
    );
    EVP_PKEY_keygen(ctx, &g_enclave_privkey);
    EVP_PKEY_CTX_free(ctx);

    /* Export public key to PEM for REPORTDATA */
    BIO *bio = BIO_new(BIO_s_mem());
    PEM_write_bio_PUBKEY(bio, g_enclave_privkey);
    g_enclave_pubkey_pem_len = BIO_read(
        bio,
        g_enclave_pubkey_pem,
        sizeof(g_enclave_pubkey_pem)
    );
    BIO_free(bio);

    return ret;
}

/**
 * ecall_get_pubkey_hash: Returns SHA-512 of enclave
 * public key for embedding in REPORTDATA before quote.
 */
sgx_status_t ecall_get_pubkey_hash(
    uint8_t out_hash[64]
) {
    sgx_sha512_msg(
        g_enclave_pubkey_pem,
        (uint32_t)g_enclave_pubkey_pem_len,
        (sgx_sha512_hash_t *)out_hash
    );
    return SGX_SUCCESS;
}

/**
 * ecall_process_job: Main job processing entry point.
```

```
 * Receives encrypted payload, decrypts, executes,
 * and returns encrypted result.
 */
sgx_status_t ecall_process_job(
    const uint8_t *enc_payload, size_t enc_len,
    const uint8_t *user_pubkey, size_t pubkey_len,
    uint8_t *enc_result, size_t *result_len
) {
    /* Step 1: ECDH key agreement with user ephemeral key */
    EVP_PKEY *user_pkey = d2i_PUBKEY(
        NULL, &user_pubkey, pubkey_len
    );
    EVP_PKEY_CTX *dh_ctx = EVP_PKEY_CTX_new(
        g_enclave_privkey, NULL
    );
    EVP_PKEY_derive_init(dh_ctx);
    EVP_PKEY_derive_set_peer(dh_ctx, user_pkey);

    size_t shared_len = 0;
    EVP_PKEY_derive(dh_ctx, NULL, &shared_len);
    uint8_t *shared = malloc(shared_len);
    EVP_PKEY_derive(dh_ctx, shared, &shared_len);

    /* Step 2: HKDF to derive content encryption key */
    uint8_t key[32]; /* AES-256 key */
    HKDF(key, 32, EVP_sha256(),
         shared, shared_len,
         NULL, 0,  /* no salt */
         (const uint8_t *)"paperclip-workload", 18);

    /* Step 3: AES-256-GCM decryption */
    /* ... AES-GCM decrypt enc_payload -> plaintext ... */

    /* Step 4: Execute workload in WASM sandbox */
    /* ... sandbox execution ... */

    /* Step 5: Encrypt result under user public key */
    /* ... ECDH + AES-GCM encrypt -> enc_result ... */

    free(shared);
    EVP_PKEY_free(user_pkey);
    EVP_PKEY_CTX_free(dh_ctx);
    return SGX_SUCCESS;
}
```

Listing 15: SGX enclave initialization: key generation and attestation report data preparation.

This enclave implementation demonstrates how the cryptographic protocol described in Section 6 maps to concrete SGX enclave code. The private key `g_enclave_privkey` is generated inside the enclave on the first `ecall_init_enclave` call and never leaves the EPC. The public key is exported to PEM format for inclusion in the attestation quote's REPORTDATA field, binding the encryption key to the enclave's measured identity.

B Supplementary Protocol Diagrams

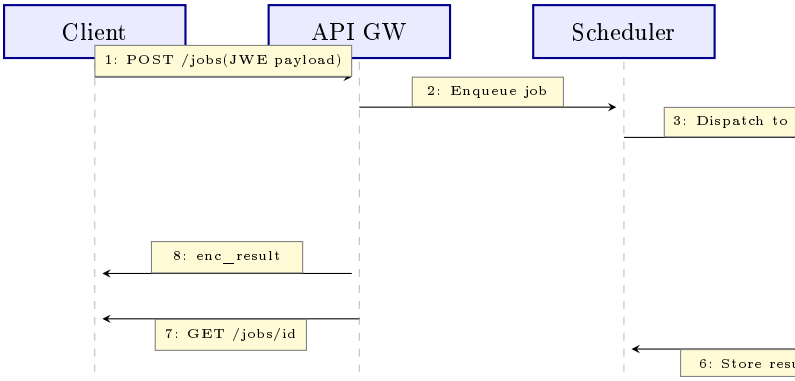

Figure 3: Full job lifecycle message sequence diagram showing all inter-service communication from job submission to encrypted result retrieval.

References

[1] V. Costan and S. Devadas, "Intel SGX Explained," *IACR Cryptology ePrint Archive*, 2016.

[2] D. Kaplan, J. Powell, and T. Woller, "AMD Memory Encryption," AMD White Paper, 2016.

[3] AMD, "AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More," AMD White Paper, 2020.

[4] ARM, "Arm Confidential Compute Architecture (CCA)," ARM Developer Documentation, 2021.

[5] NVIDIA, "NVIDIA H100 Tensor Core GPU Architecture," NVIDIA Technical Report, 2023.

[6] L. Zhu, Z. Liu, and S. Han, "Deep Leakage from Gradients," in *Proc. NeurIPS*, 2019.

[7] R. Gilad-Bachrach *et al.*, "CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy," in *Proc. ICML*, 2016.

[8] P. Mohassel and Y. Zhang, "SecureML: A System for Scalable Privacy-Preserving Machine Learning," in *Proc. IEEE S&P*, 2017.

[9] C. Dwork *et al.*, "Calibrating Noise to Sensitivity in Private Data Analysis," in *Proc. TCC*, 2006.

[10] B. McMahan *et al.*, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. AISTATS*, 2017.

[11] R. Shokri *et al.*, "Membership Inference Attacks Against Machine Learning Models," in *Proc. IEEE S&P*, 2017.

[12] S. Johnson *et al.*, "Intel Software Guard Extensions: EPID Provisioning and Attestation Services," Intel White Paper, 2016.

[13] F. Tramèr and D. Boneh, "Slalom: Fast, Verifiable and Private Execution of Neural Networks in Trusted Hardware," in *Proc. ICLR*, 2019.

[14] A. Baumann *et al.*, "Shielding Applications from an Untrusted Cloud with Haven," in *Proc. OSDI*, 2015.

[15] C. C. Tsai *et al.*, "Graphene-SGX: A Practical Library OS for Unmodified Applications on SGX," in *Proc. USENIX ATC*, 2017.

[16] Gramine Project, "Gramine: A Library OS for Running Unmodified Applications in SGX," `https://gramineproject.io`, 2023.

[17] F. Schuster *et al.*, "VC3: Trustworthy Data Analytics in the Cloud Using SGX," in *Proc. IEEE S&P*, 2015.

[18] Microsoft, "Open Enclave SDK," `https://openenclave.io`, 2023.

[19] M. Orenbach *et al.*, "Eleos: ExitLess OS Services for SGX Enclaves," in *Proc. EuroSys*, 2017.

[20] P. Kocher *et al.*, "Spectre Attacks: Exploiting Speculative Execution," *Commun. ACM*, 2020.

[21] M. Lipp *et al.*, "Meltdown: Reading Kernel Memory from User Space," in *Proc. USENIX Security*, 2018.

[22] J. Van Bulck *et al.*, "Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution," in *Proc. USENIX Security*, 2018.

[23] J. Van Bulck *et al.*, "SGAxe: How SGX Fails in Practice," in *Proc. IEEE S&P*, 2020.

[24] A. Moghimi *et al.*, "CacheZoom: How SGX Amplifies the Power of Cache Attacks," in *Proc. CHES*, 2017.

[25] K. Murdock *et al.*, "Plundervolt: Software-Based Fault Injection Attacks Against Intel SGX," in *Proc. IEEE S&P*, 2020.

[26] Z. Chen *et al.*, "VoltPillager: Hardware-Based Fault Injection Attacks Against Intel SGX Enclaves Using the SVID Voltage Scaling Interface," in *Proc. USENIX Security*, 2021.

[27] M. Schwarz *et al.*, "ZombieLoad: Cross-Privilege-Boundary Data Sampling," in *Proc. CCS*, 2019.

[28] W. Wang *et al.*, "Leaky Cauldron on the Dark Land: Understanding Memory Side-Channel Hazards in SGX," in *Proc. CCS*, 2017.

[29] A. Kwong *et al.*, "RAMBleed: Reading Bits in Memory Without Accessing Them," in *Proc. IEEE S&P*, 2020.

[30] S. Pinto and N. Santos, "Demystifying ARM TrustZone: A Comprehensive Survey," *ACM Comput. Surv.*, 2019.

[31] C. Juvekar *et al.*, "GAZELLE: A Low Latency Framework for Secure Neural Network Inference," in *Proc. USENIX Security*, 2018.

[32] B. D. Rouhani *et al.*, "DeepSecure: Scalable Provably-Secure Deep Learning," in *Proc. DAC*, 2018.

[33] P. Mishra *et al.*, "Delphi: A Cryptographic Inference Service for Neural Networks," in *Proc. USENIX Security*, 2020.

[34] C. Gentry, "A Fully Homomorphic Encryption Scheme," Stanford University PhD Thesis, 2009.

[35] I. Chillotti *et al.*, "TFHE: Fast Bootstrapping over the Torus," *J. Cryptol.*, 2020.

[36] A. C. Yao, "How to Generate and Exchange Secrets," in *Proc. FOCS*, 1986.

[37] T. Ryffel *et al.*, "A Generic Framework for Privacy Preserving Deep Learning," in *NeurIPS Workshop on PPML*, 2018.

[38] K. Bonawitz *et al.*, "Towards Federated Learning at Scale: System Design," in *Proc. MLSys*, 2019.

[39] WeBank, "FATE: Federated AI Technology Enabler," `https://fate.fedai.org`, 2023.

[40] T. Hunt *et al.*, "Telekine: Secure Computing with Cloud GPUs," in *Proc. NSDI*, 2020.

[41] Microsoft, "Azure Confidential Computing," `https://azure.microsoft.com/confidential-computing`, 2023.

[42] Amazon, "AWS Nitro Enclaves," `https://aws.amazon.com/ec2/nitro/nitro-enclaves`, 2023.

[43] Google, "Confidential VMs," `https://cloud.google.com/confidential-computing`, 2023.

[44] Akash Network, "Akash: The Open Cloud Network," `https://akash.network`, 2023.

[45] Golem Network, "Golem: The World's Decentralized Computer," `https://golem.network`, 2023.

[46] iExec, "iExec: Decentralized Cloud Computing," `https://iex.ec`, 2023.

[47] Gensyn, "Gensyn: Decentralized Machine Learning," `https://gensyn.ai`, 2023.

[48] Bittensor, "Bittensor: A Peer-to-Peer Intelligence Market," `https://bittensor.com`, 2023.

[49] A. Verma *et al.*, "Large-Scale Cluster Management at Google with Borg," in *Proc. EuroSys*, 2015.

[50] M. Schwarzkopf *et al.*, "Omega: Flexible, Scalable Schedulers for Large Compute Clusters," in *Proc. EuroSys*, 2013.

[51] B. Hindman *et al.*, "Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center," in *Proc. NSDI*, 2011.

[52] B. Burns *et al.*, "Borg, Omega, and Kubernetes," *ACM Queue*, 2016.

[53] P. Moritz *et al.*, "Ray: A Distributed Framework for Emerging AI Applications," in *Proc. OSDI*, 2018.

[54] M. Zaharia *et al.*, "Delay Scheduling: A Simple Technique for Achieving Locality and Fairness in Cluster Scheduling," in *Proc. EuroSys*, 2010.

[55] S. Tang *et al.*, "Completing Your Tasks on Time: Plan Your Schedule Before Running," in *Proc. EUROSYS*, 2013.

[56] F. Zhang *et al.*, "Town Crier: An Authenticated Data Feed for Smart Contracts," in *Proc. CCS*, 2016.

[57] R. Cheng *et al.*, "Ekiden: A Platform for Confidentiality-Preserving, Trustworthy, and Performant Smart Contract Execution," in *Proc. EuroS&P*, 2019.

[58] M. Li *et al.*, "Confidential and Attested Execution of Machine Learning on AMD SEV-SNP," *arXiv*, 2021.

[59] J. Menetrey *et al.*, "Attestation Mechanisms for Trusted Execution Environments Demystified," in *Proc. ICDCS*, 2021.

[60] S. Sasy *et al.*, "ZeroTrace: Oblivious Memory Primitives from Intel SGX," in *Proc. NDSS*, 2018.

[61] A. W. Appel, "Verified Software Toolchain," in *Proc. ESOP*, 2011.

[62] J. Protzenko *et al.*, "Verified Low-Level Programming Embedded in F*," in *Proc. ICFP*, 2017.

[63] Enarx Project, "Enarx: Run Your Code in a Secure Enclave," `https://enarx.dev`, 2023.

[64] Render Network, "Render Network: Decentralized GPU Rendering," `https://rendernetwork.com`, 2023.

[65] P. L. Aublin *et al.*, "Library Operating Systems on Untrusted Platforms: Enabling Trustworthy Distributed Computing," *IEEE Trans. Dependable Sec. Comput.*, 2017.

[66] Fortanix, "Fortanix Runtime Encryption Platform," `https://fortanix.com`, 2023.

[67] Edgeless Systems, "Marblerun: A Control Plane for Confidential Computing," `https://edgeless.systems`, 2023.

[68] Anjuna Security, "Anjuna Confidential Computing Platform," `https://anjuna.io`, 2023.

[69] I. Kolosov *et al.*, "TEEMS: Trusted Execution Environments for Mobile Systems," in *Proc. NDSS*, 2021.

[70] D. Acemoglu *et al.*, "Power and Progress: Our Thousand-Year Struggle Over Technology and Prosperity," Basic Books, 2023.